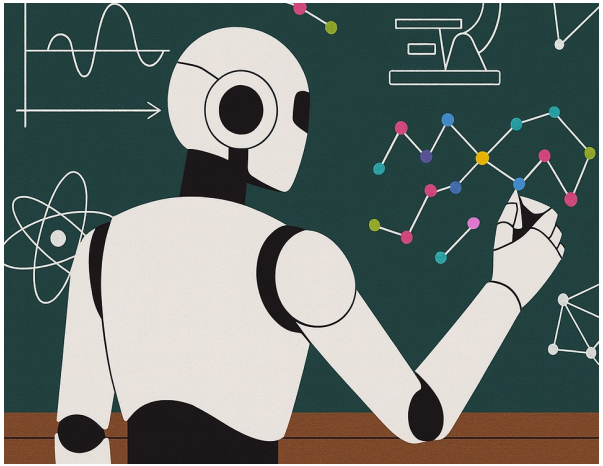
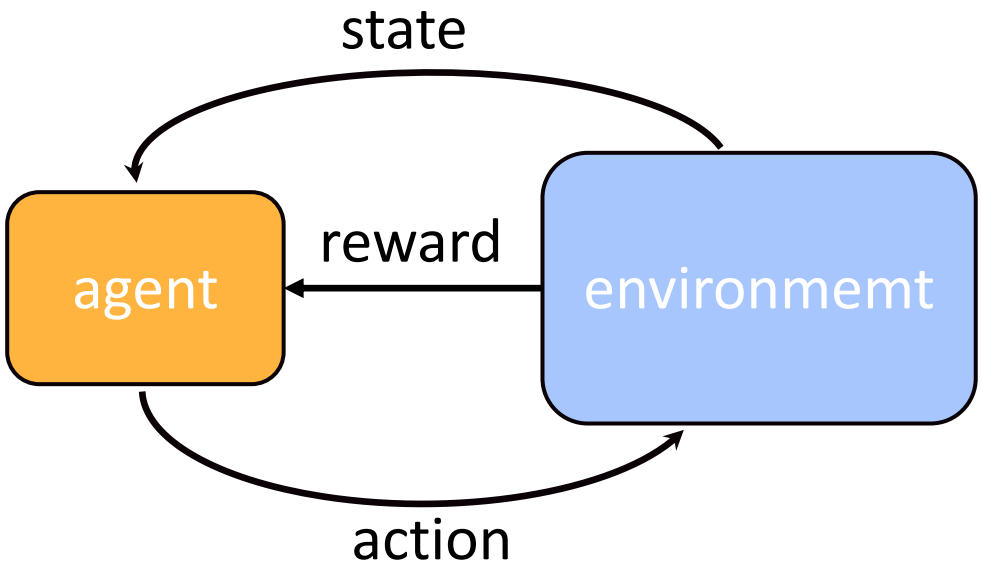
If you have any questions, feel free to reach out!

xuemei.gu@uni-jena.de

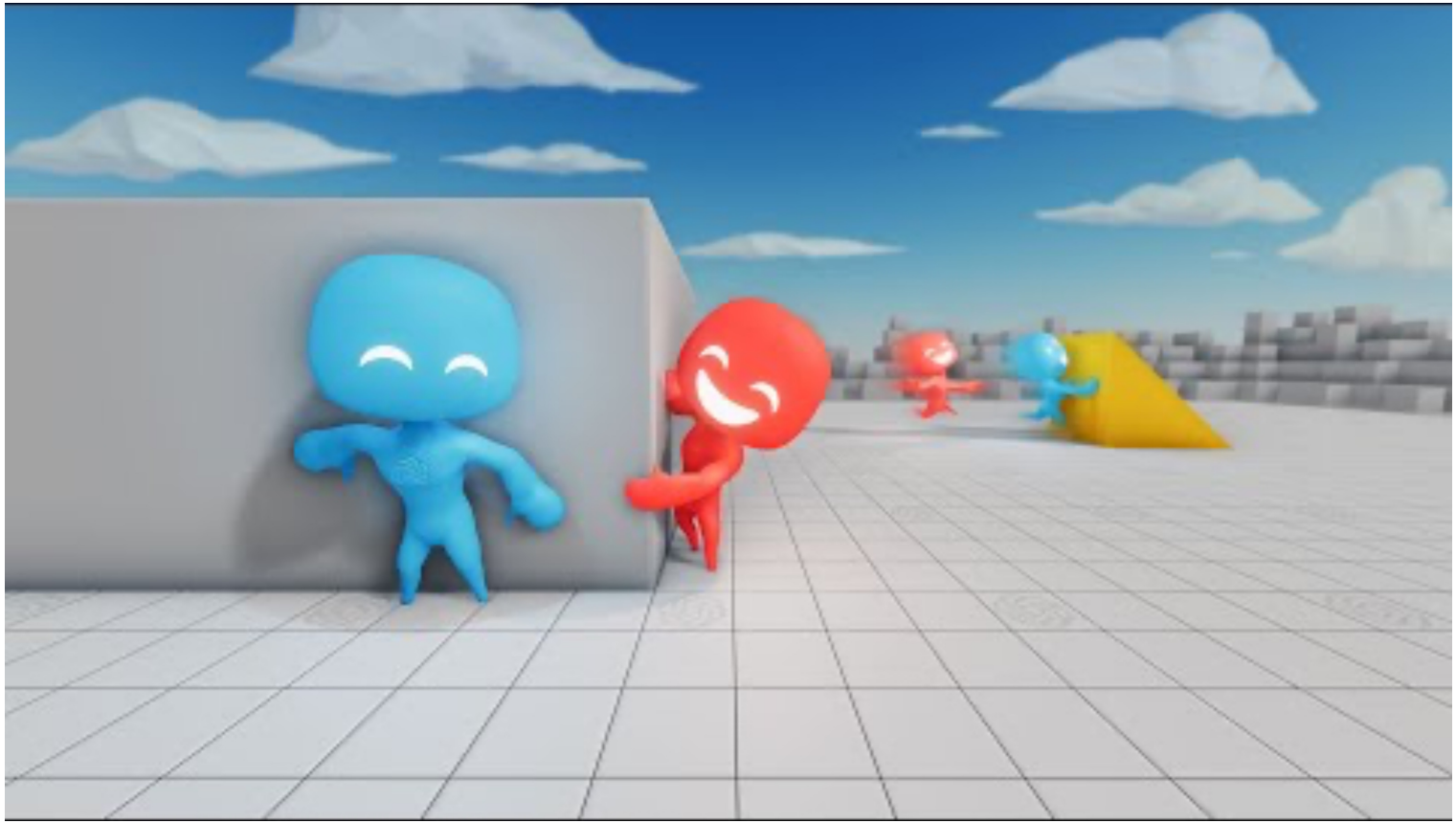
AI for Quantum Physics



Winter semester 2025/26



Reinforcement Learning



https://www.youtube.com/watch?v=kopoLzvh5jY

Class of Learning Problems

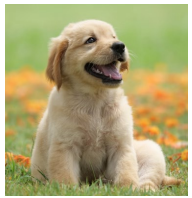
Supervised learning

Data: (x, y)

x is data, y is label

Learn function to map

$x \rightarrow y$



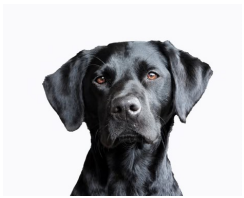
This is a dog

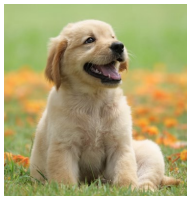
Unsupervised Learning

Data: x

x is data, **no label**

Learn hidden or underlying structure





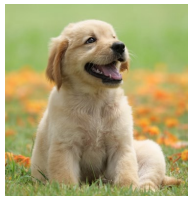
This thing is like the other thing

Reinforcement learning

Data:

State, action

Learn how to take actions in order to maximize reward



Have this thing because it will make you happy

What is Reinforcement Learning?

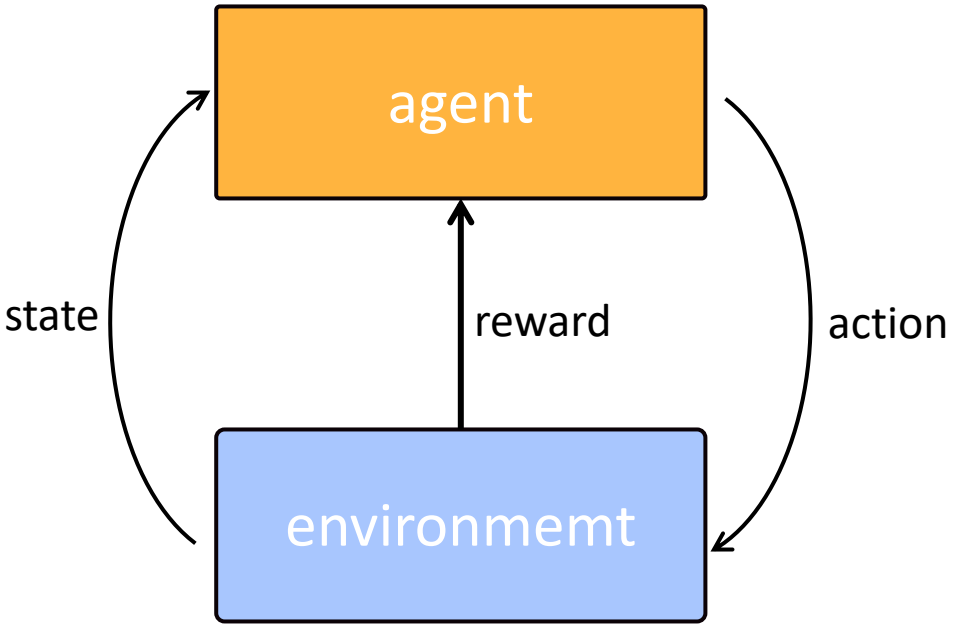
Good for problems involving an agent interacting with an environment, which provides numeric reward signals





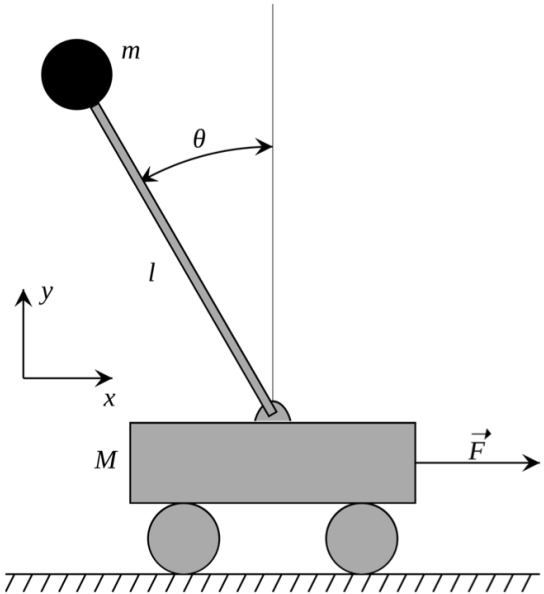
Learning **by trial and error**
Numerical reward is often delayed

Reinforcement Learning: key concepts & terminology



- At each step t the agent:
  - Executes action a_t
  - Receives observation o_t
  - Receives scalar reward r_t
- The environment:
  - Receives action a_t
  - Emits observation o_{t+1}
  - Emits scalar reward r_{t+1}

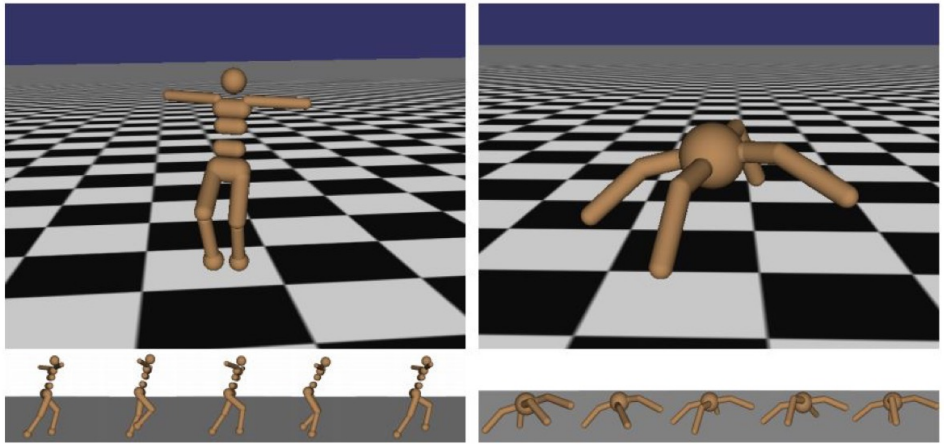
Cart-Pole Problem



Objective: Balance a pole on top of a movable cart

State: angle, angular speed, position, horizontal velocity
Action: horizontal force applied on the cart
Reward: 1 at each time step if the pole is upright

Robot Locomotion



Objective: Make the robot move forward

State: Angle and position of the joints
Action: Torques applied on joints
Reward: 1 at each time step upright + forward movement

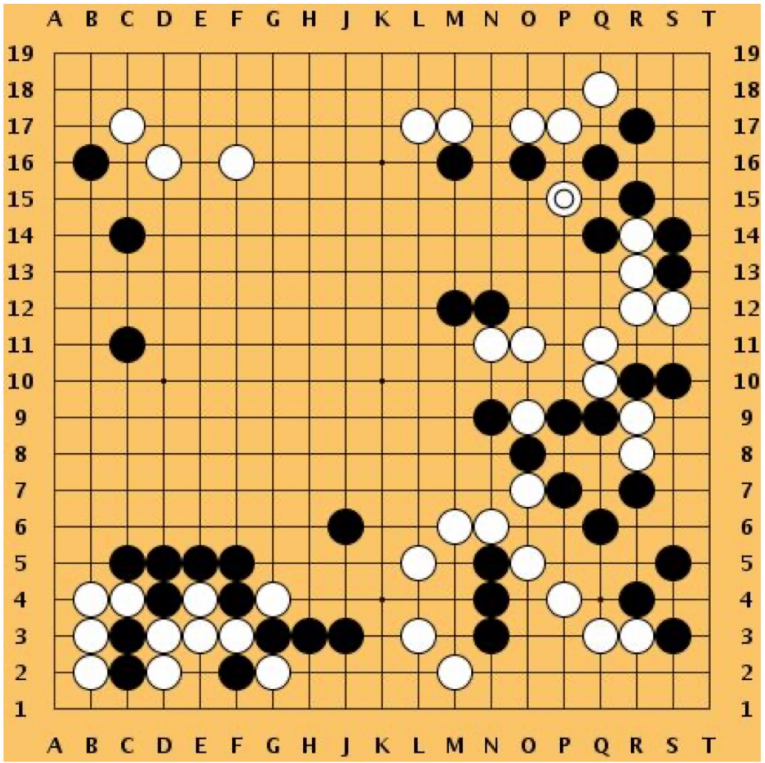
Atari Games



Objective: Complete the game with the highest score

State: Raw pixel inputs of the game state
Action: Game controls e.g. Left, Right, Up, Down
Reward: Score increase/decrease at each time step

Go

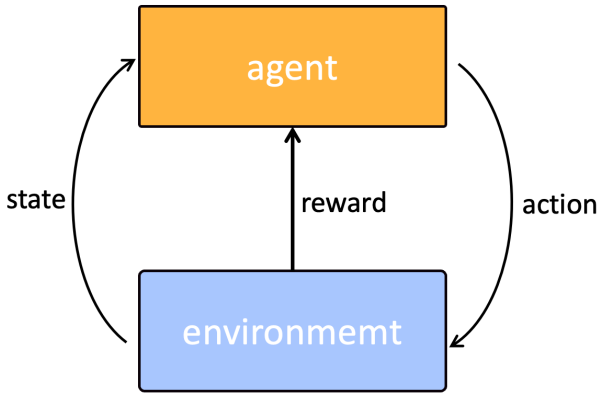


Objective: Win the game!

State: Position of all pieces
Action: Where to put the next piece down
Reward: 1 if win at the end of the game, 0 otherwise

Reinforcement Learning: key concepts & terminology

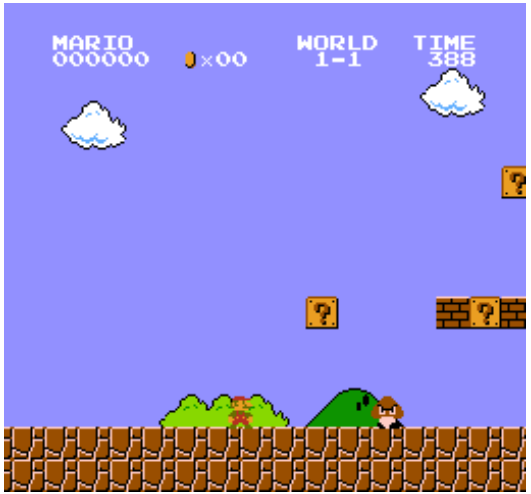
The State Space is the complete set of all possible configurations of the environment, while the Observation Space is the specific, potentially partial or noisy information the agent receives about that state, defining what the agent actually perceives to make decisions



States/Observations Space

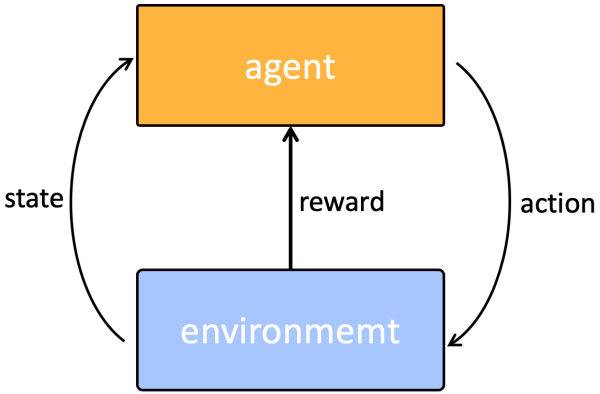


complete description of the state of the world (no hidden information)



partial description of the state of the world

Reinforcement Learning: key concepts & terminology



Action Space

the set of all possible actions in an environment

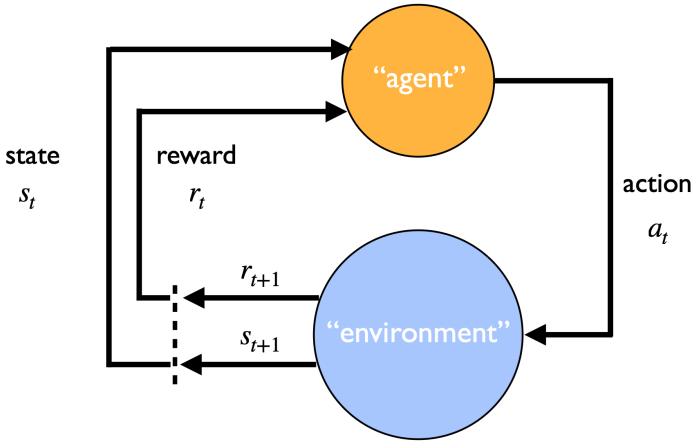


Discrete: finite number of possible actions



Continuous: infinite number of possible actions

Reinforcement Learning: Goals and rewards



Goal:

Maximize total amount of reward received

Maximize expected value of cumulative sum of rewards over trajectory (return)

$$\mathbb{E}\left\{\sum_{t=0}^{T} r_t\right\}$$

$$R(\tau) = r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4} + \dots$$

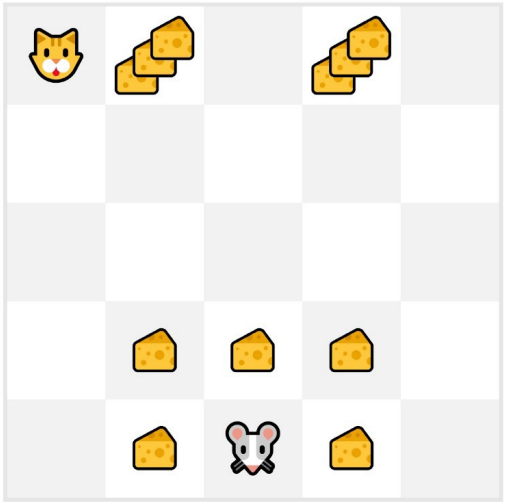
Return: cumulative reward

telling the agent what is to be achieved, not how to do it

Trajectory

Sequence of states and actions

Reinforcement Learning: key concepts & terminology



how much it values long-term gains over short-term ones

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

Return: cumulative reward

Gamma: discount rate

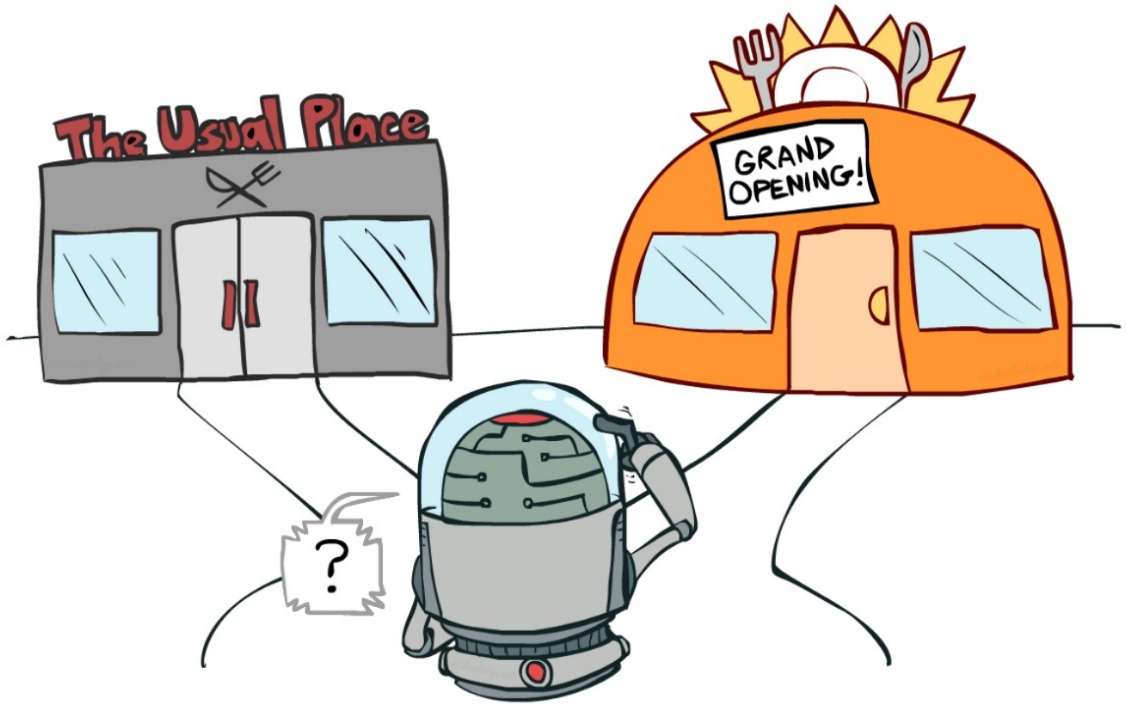
Trajectory
Sequence of states and actions

Exploitation:

using known information to maximize the reward.

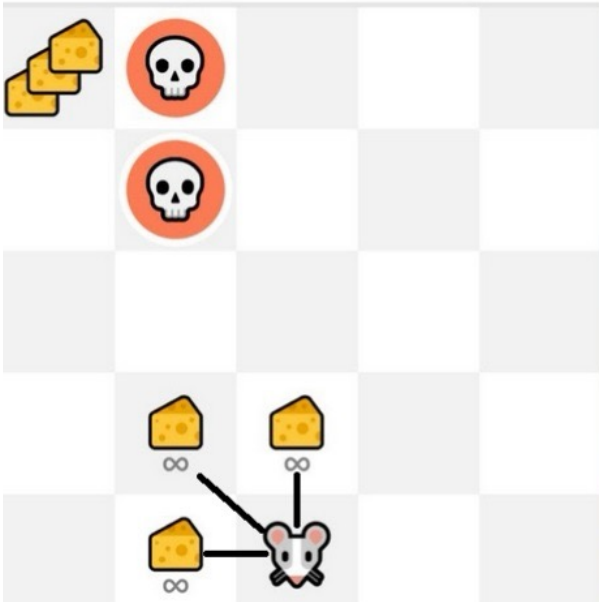
Exploration:

trying random actions in order to find more information about the environment.



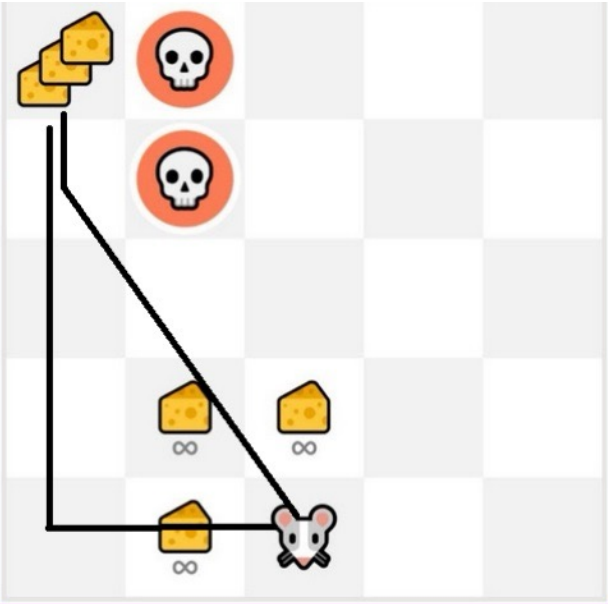
Exploitation:

using known information to maximize the reward.

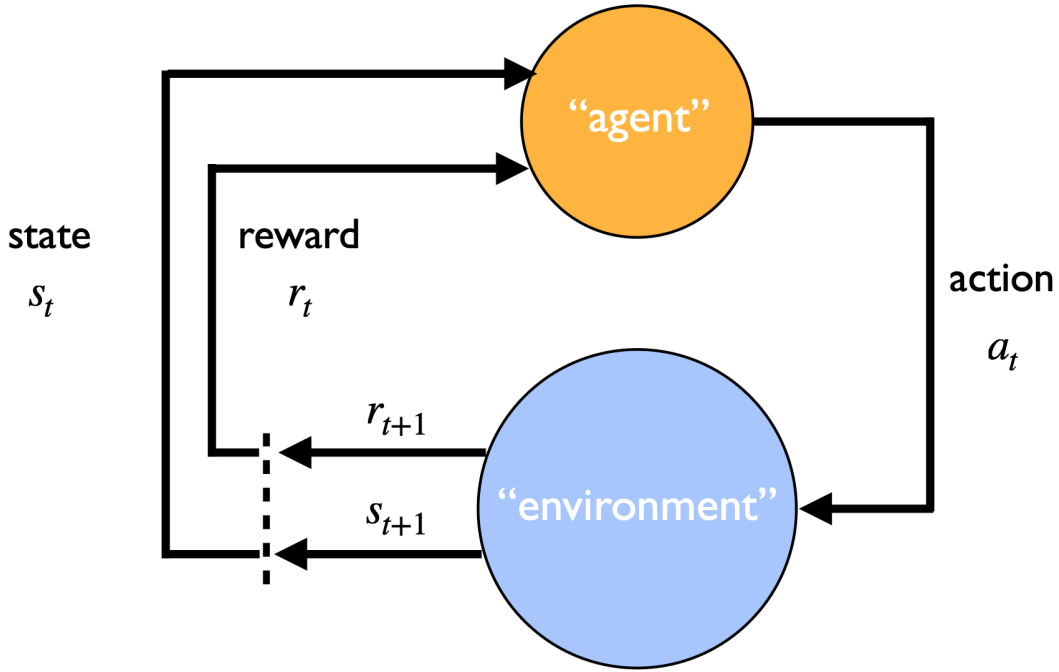


Exploration:

trying random actions in order to find more information about the environment.



How can we mathematically formalize the RL problem?



Markov Decision Process (MDP)

mathematically formalize the RL problem

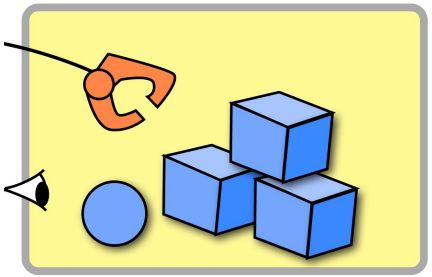
Defined by: $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathbb{P}, \gamma)$

$\mathcal{S}$: set of possible states
$\mathcal{A}$: set of possible actions
$\mathcal{R}$: distribution of reward given (state, action) pair
$\mathbb{P}$: transition probability i.e. distribution over next state given (state, action) pair
γ : discount factor

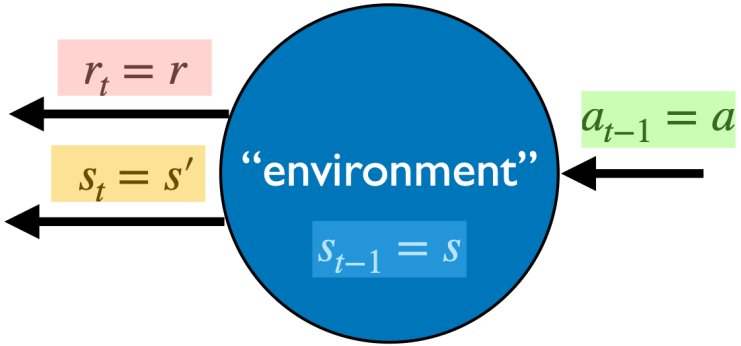
trajectory: $S_0, A_0, R_1, S_1 \ldots$

State Action Reward Next State

Markov property: Current state completely characterizes the state of the world



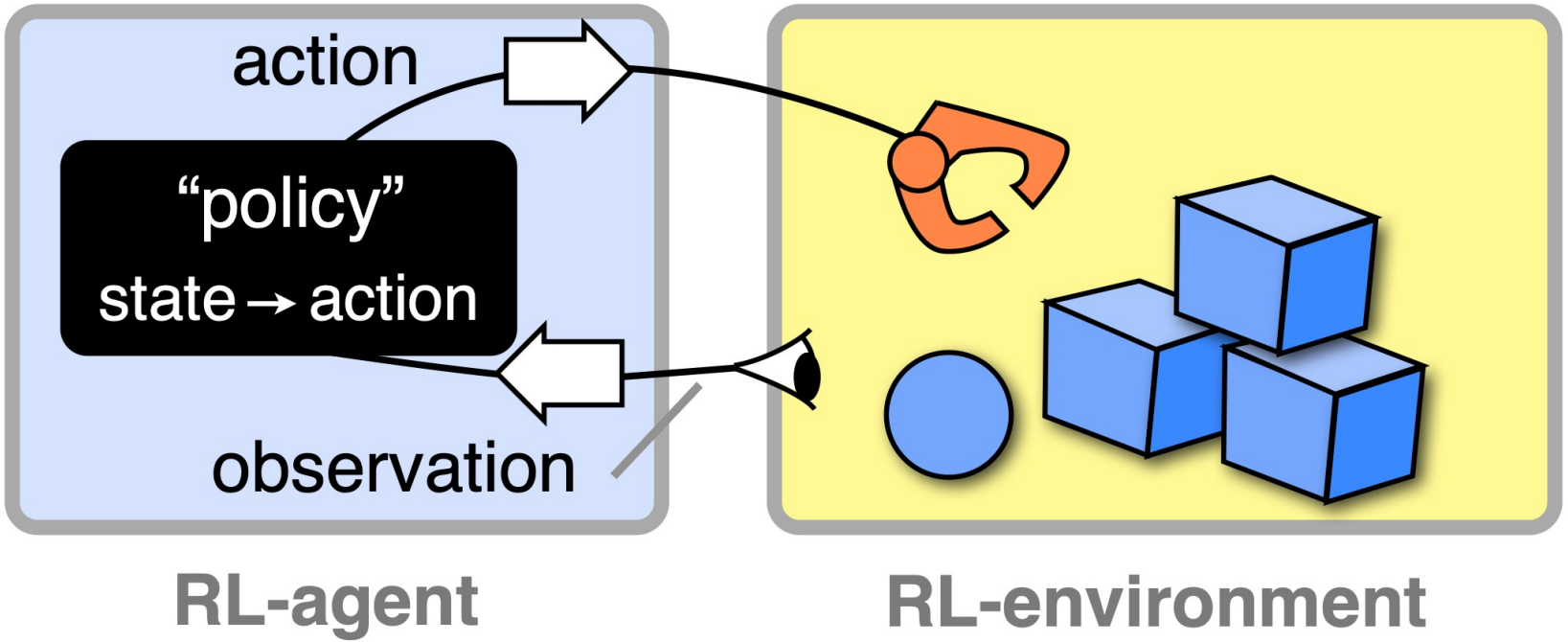
RL-environment



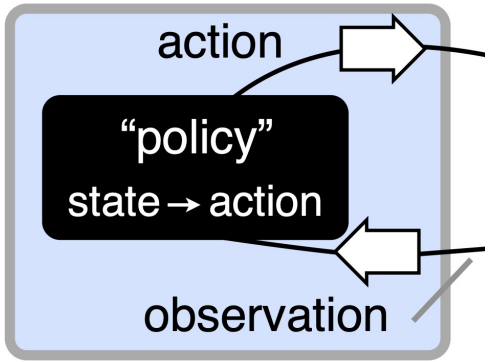
$$p(s', r \mid s, a) \equiv \Pr\{s_t = s', r_t = r \mid s_{t-1} = s, a_{t-1} = a\}$$

p defines the *dynamics* of the MDP:

Random variables r_t and s_t have well-defined probabilities dependent only on preceding state and action


action
"policy"
state → action
observation
RL-agent
RL-environment

Learns a Policy

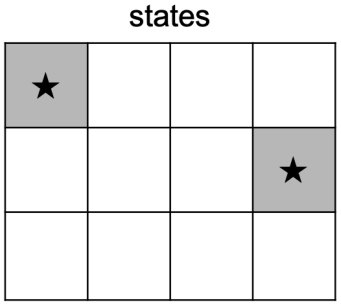


RL-agent

actions = {

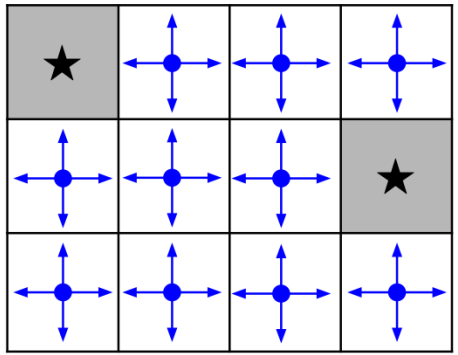
1. right
2. left
3. up
4. down

}

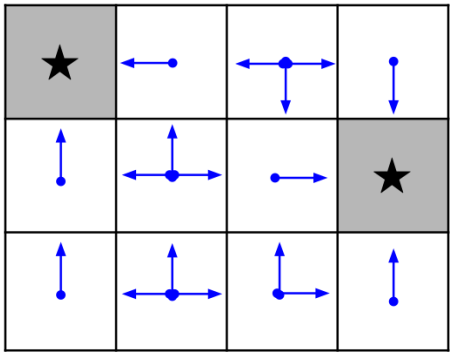


Set a negative "reward" for each transition (e.g. r = -1)

Objective: reach one of terminal states (greyed out) in least number of actions

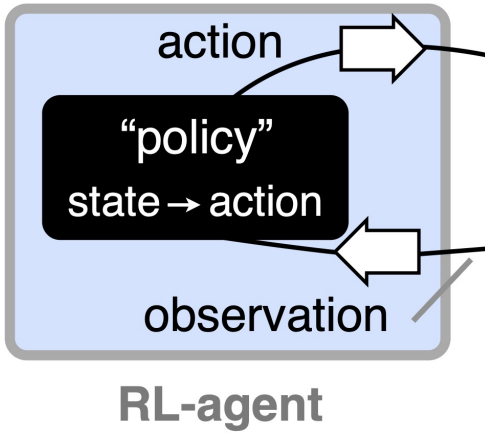


Random policy



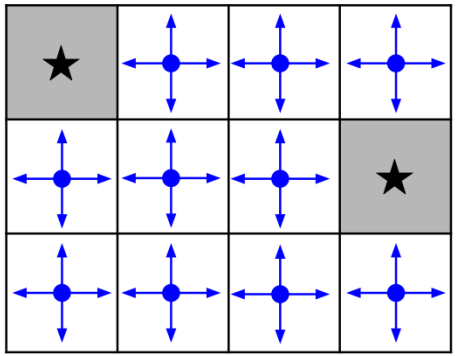
Optimal policy

Policy — How does the agent behave?

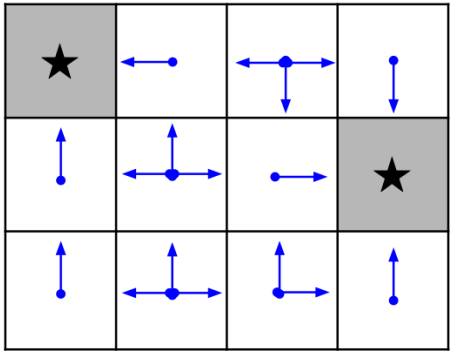


Policy: Mapping from states to probabilities of selecting each possible action

$$\pi_\theta(a_t \mid s_t)$$

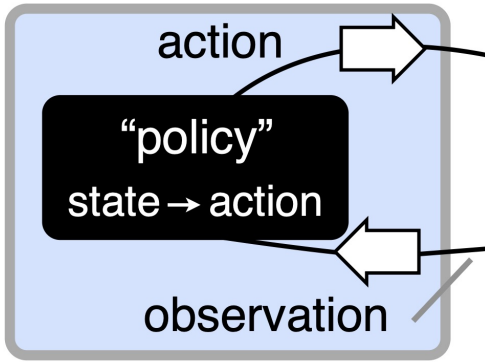


Random policy



Optimal policy

Policy — How does the agent behave?



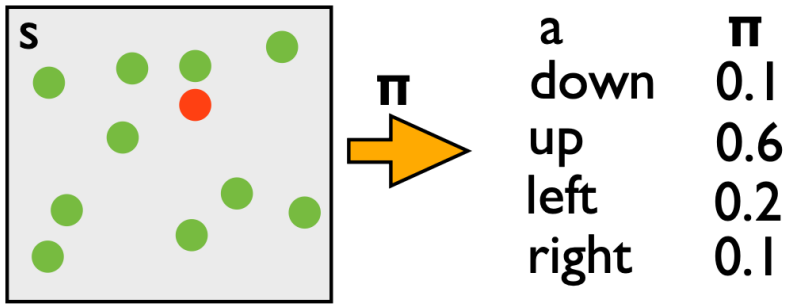
RL-agent

Policy: Mapping from states to probabilities of selecting each possible action

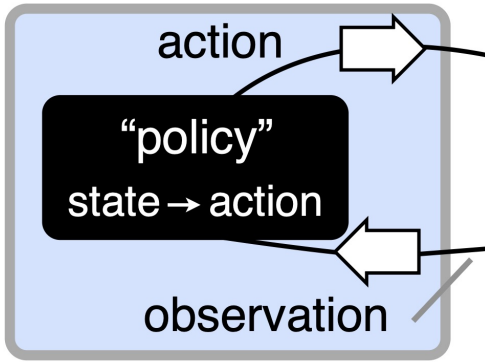
$$\pi_\theta(a_t \mid s_t)$$

Deterministic policy: $a = \pi(s)$

Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$



Experiences
Rewards

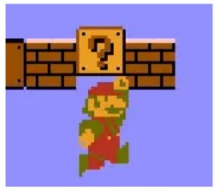


RL-agent

How good is a given state? $r_t = R(s_t)$

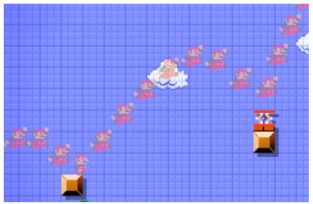


How good is a state-action pair? $r_t = R(s_t, a_t)$



How good is a *trajectory* $\tau = (s_0, a_0, s_1, a_1, \ldots)$:

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^k r_t \text{ where } \gamma \in [0,1] \text{ is the temporal discount}$$



Value: Functions that estimate how good it is for the agent to be in a given state (assuming policy π_θ)

State-value function: Expected return when starting in s and following π_θ thereafter:

$$v_\pi(s) \equiv \mathbb{E}_\pi \left[\sum_{k=0} r_{t+k+1} \,|\, s_t = s \right]$$

Value Function

$$v_\pi(s) = \sum_a q_\pi(s,a)\pi_\theta(a \,|\, s)$$

Action-value function: Expected return when starting in s, taking action a, and following π_θ thereafter:

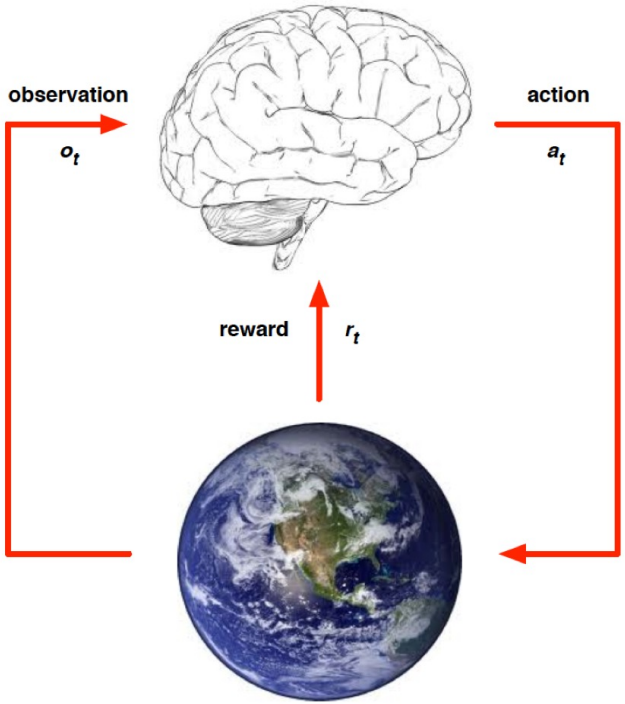
$$q_\pi(s,a) \equiv \mathbb{E}_\pi \left[\sum_{k=0} r_{t+k+1} \,|\, s_t = s,\ a_t = a \right]$$

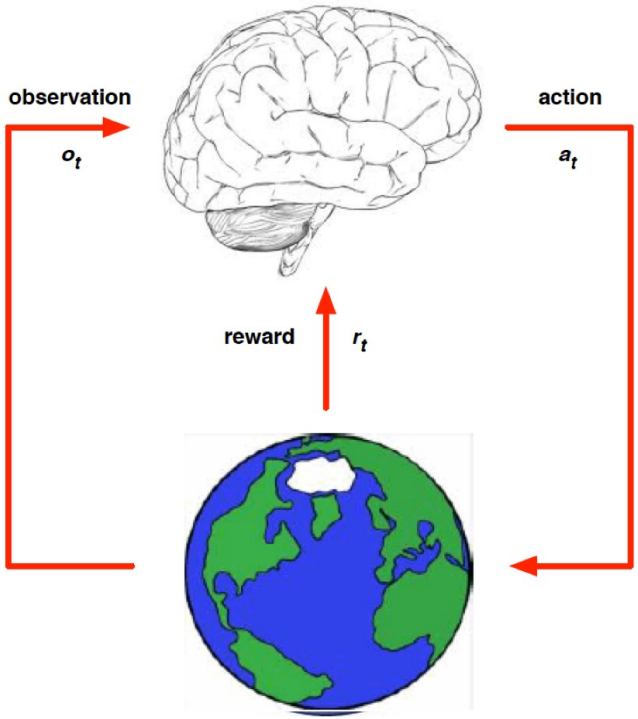
Q-value Function

$$q_\pi(s,a) = \sum_{s',r} p(s', r \,|\, s, a)(r + v_\pi(s'))$$

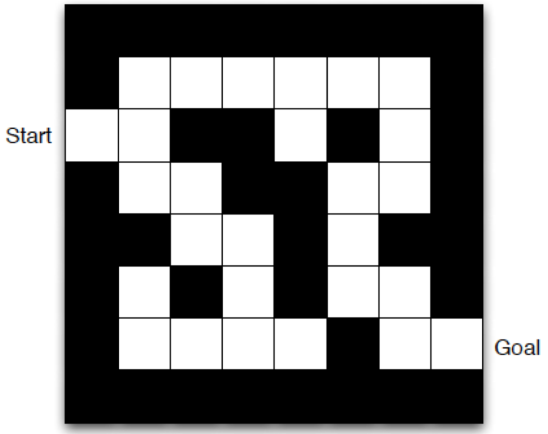
Model — Agent's representation of the environment

Model predicts what the world will do next

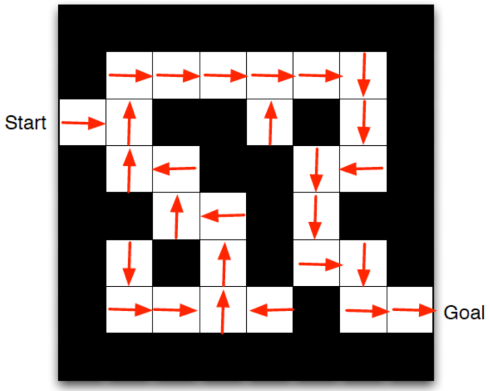




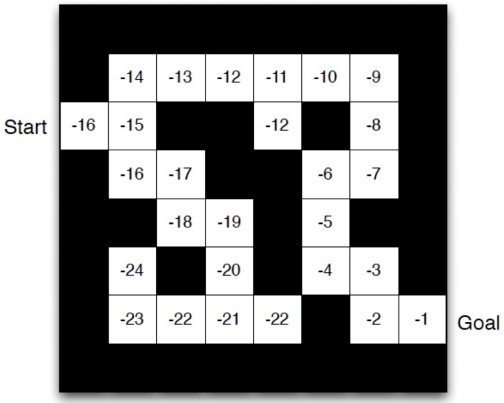
* slide from David Silver



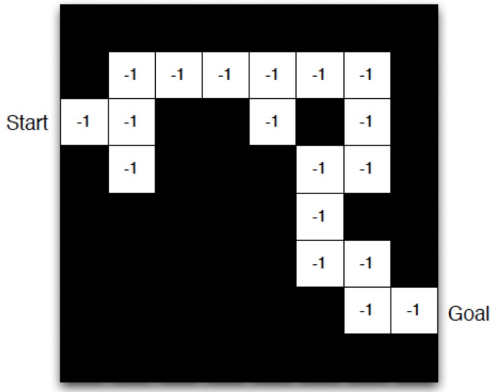
Reward: -1 per time-step
Actions: N, E, S, W
States: Agent's location



Arrows represent a policy $\pi(s)$ for each state s



Numbers represent value $v_\pi(s)$ of each state s



Grid layout represents transition model

Numbers represent the immediate reward for each state (same for all states)

Reinforcement Learning Approaches

Model-free RL

Value-based RL

- Estimate the optimal action-value function $Q^*(s, a)$
- No policy (implicit)

Policy-based RL

- Search directly for the optima policy π^*
- No value function

Actor-critic RL

- Value function
- Policy function

Model-based RL

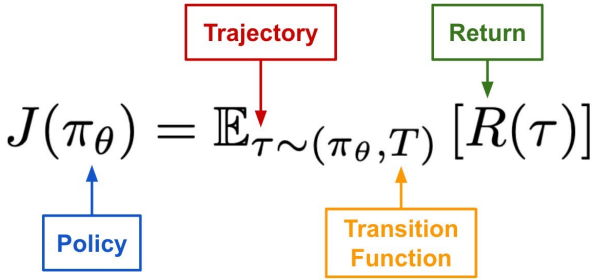
- Build a model of the world
- Plan (e.g., by look-ahead) using model

* slide from Dhruv Batra

Policy Gradient Methods

Can we learn a policy directly, e.g. finding the best policy from a collection of policies?

For each policy, define its value



$$J(\theta) = \mathbb{E}\left[\sum_{t \geq 0} \gamma^t r_t | \pi_\theta\right]$$

Policy: Mapping from states to probabilities of selecting each possible action

$$\pi_\theta(a_t \mid s_t)$$

We want to find the optimal policy $\theta^* = \arg\max_\theta J(\theta)$

How can we do this?

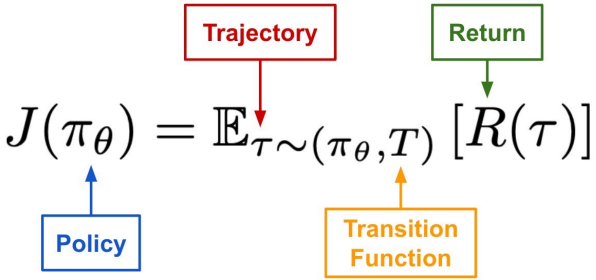
Gradient ascent on policy parameters!

Policy Gradient Methods

Gradient ascent on policy parameters!

$$\theta_{\tau+1} = \theta_\tau + \eta \boxed{\langle \nabla J(\theta_\tau) \rangle}$$

How can we compute this?



$$J(\pi_\theta) = \mathbb{E}_{\tau \sim (\pi_\theta, T)} [R(\tau)]$$

$$\begin{aligned} \nabla_\theta J(\pi_\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim (\pi_\theta, T)} [R(\tau)] \\ &= \nabla_\theta \int_\tau P(\tau|\theta) R(\tau) \\ &= \int_\tau \nabla_\theta P(\tau|\theta) R(\tau) \end{aligned}$$

Discrete case: there are a fixed, discrete number of trajectories over which we want to compute the expected return

$$\mathbb{E}_{\tau \sim (\pi_\theta, T)} [R(\tau)] = \sum_\tau P(\tau|\theta) R(\tau)$$

Continuous case: there are a potentially infinite number of trajectories over which we want to compute the expected return

$$\mathbb{E}_{\tau \sim (\pi_\theta, T)} [R(\tau)] = \int_\tau P(\tau|\theta) R(\tau)$$

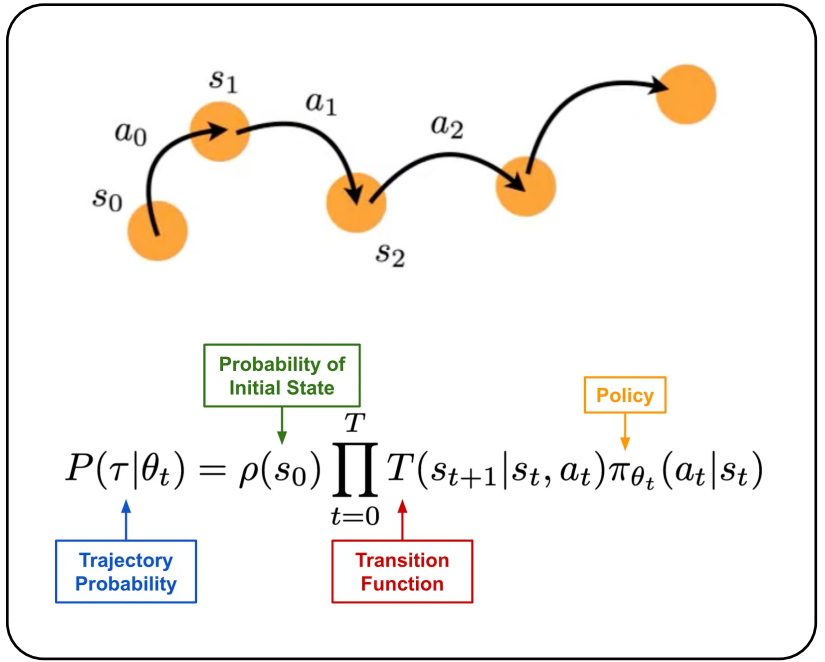
Policy Gradient Methods

$$\nabla_\theta J(\pi_\theta) = \nabla_\theta \mathbb{E}_{\tau\sim(\pi_\theta,T)}\left[R(\tau)\right]$$
$$= \nabla_\theta \int_\tau P(\tau|\theta)R(\tau)$$

Intractable! Gradient of an expectation is problematic when p depends on θ

$$= \int_\tau \nabla_\theta P(\tau|\theta)R(\tau)$$

$$\nabla_\theta p(\tau;\theta) = p(\tau;\theta)\frac{\nabla_\theta p(\tau;\theta)}{p(\tau;\theta)} = p(\tau;\theta)\nabla_\theta \log p(\tau;\theta)$$



Basic Policy Gradient

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim(\pi_\theta,T)}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)R(\tau)\right]$$

Doesn't depend on transition probabilities!

Variants of the Basic Policy Gradient

Basic Policy Gradient

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim (\pi_\theta, T)} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau) \right]$$

$$\nabla_\theta J(\pi_\theta) = \mathbb{E} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \left(\underbrace{\sum_{i=t}^{T} r_{s_i,a_i}}_{\text{Rewards after the current action}} - \underbrace{b(s_i)}_{\text{Baseline Function}} \right) \right]$$

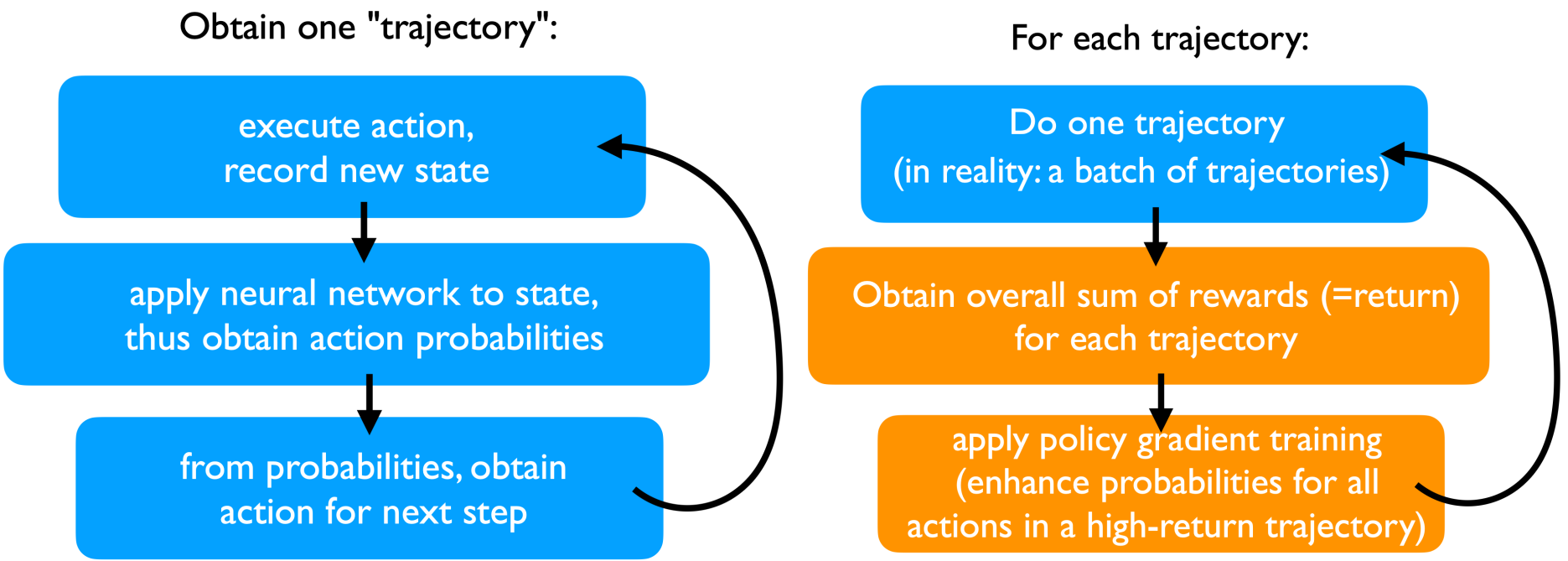
$$\nabla_\theta J(\pi_\theta) = \mathbb{E} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \underbrace{A^{\pi_\theta}(s_t, a_t)}_{\text{Advantage Function}} \right]$$

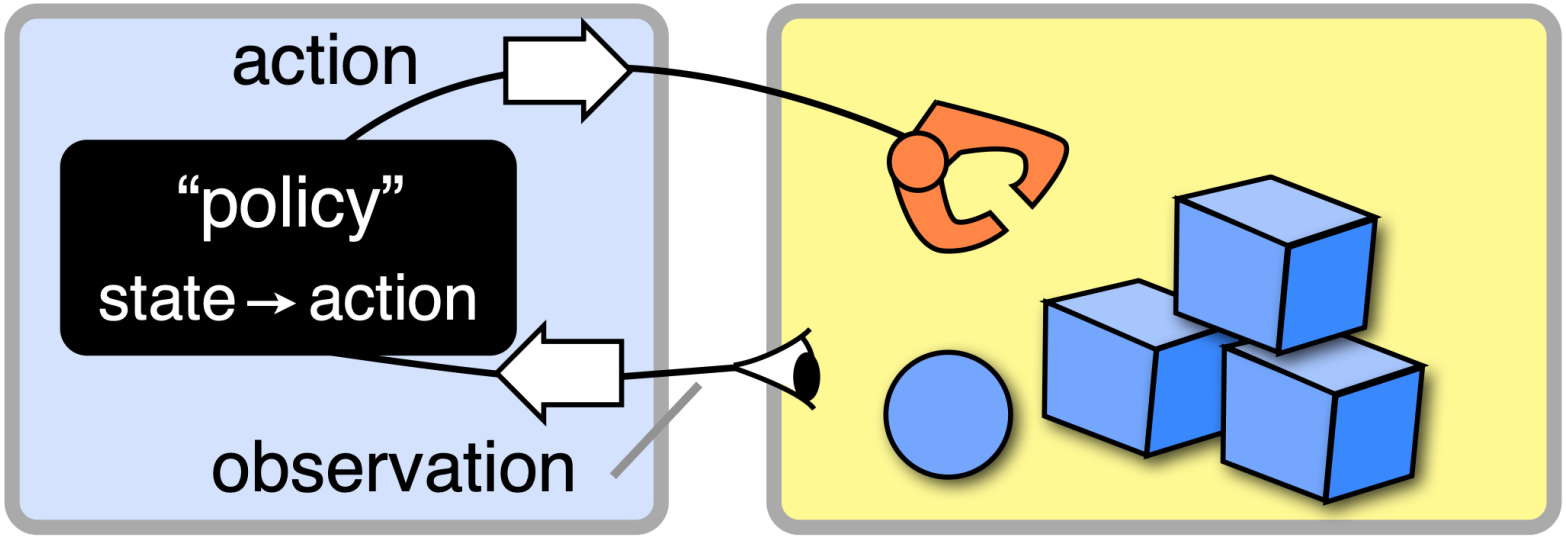
$$\nabla_\theta J(\pi_\theta) = \mathbb{E} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \hat{R}(\tau, t) \right]$$

$$\text{where } \underbrace{\hat{R}(\tau, t) = \sum_{i=t}^{T} r_{s_i,a_i}}_{\text{Rewards after the current action}}$$

Use neural nets to represent the policy π_θ

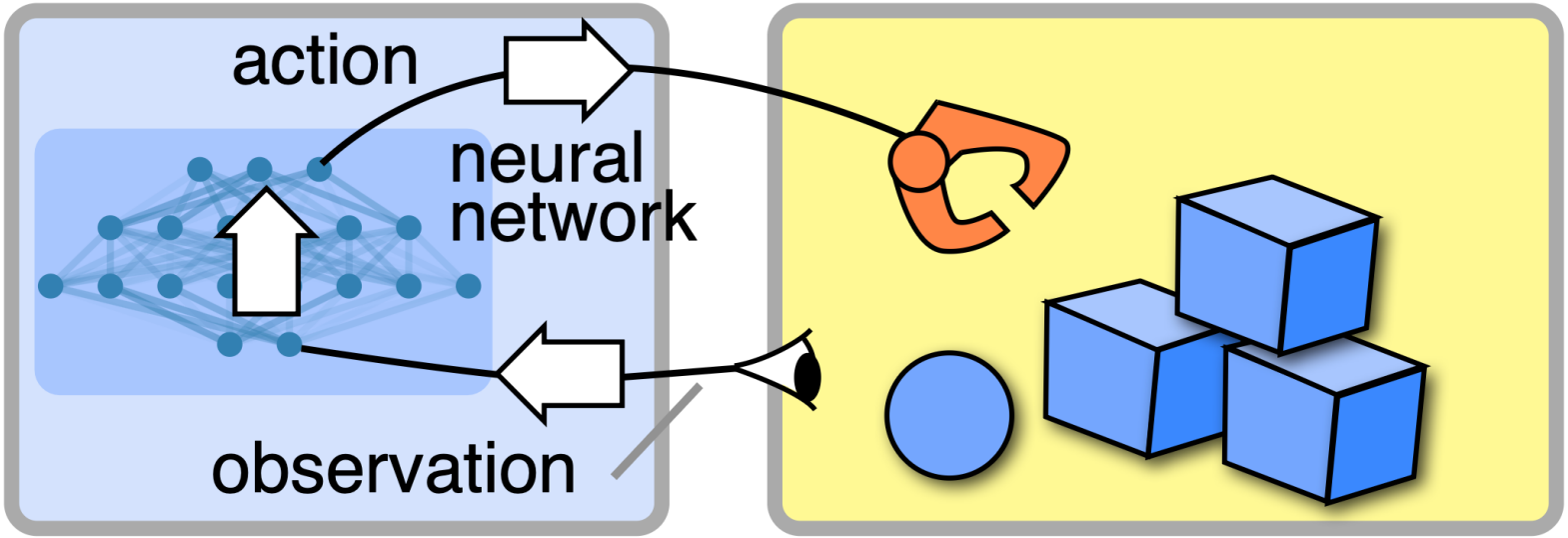
$\pi_{\theta^*} \approx \pi^*$





Policy: $\pi_\theta(a_t|s_t)$ – probability to pick action a_t given observed state s_t at time t

$$\mathcal{L}_{\text{policy}} = -\mathbb{E}\Big[\log \pi_\theta(a_t \mid s_t)\ R_t\Big]$$



Value-based RL

Action-value function: Expected return when starting in s, taking action a, and following π_θ thereafter:

$$q_\pi(s,a) \equiv \mathbb{E}_\pi \left[\sum_{k=0} r_{t+k+1} \,|\, s_t = s,\ a_t = a \right]$$

Q-value Function

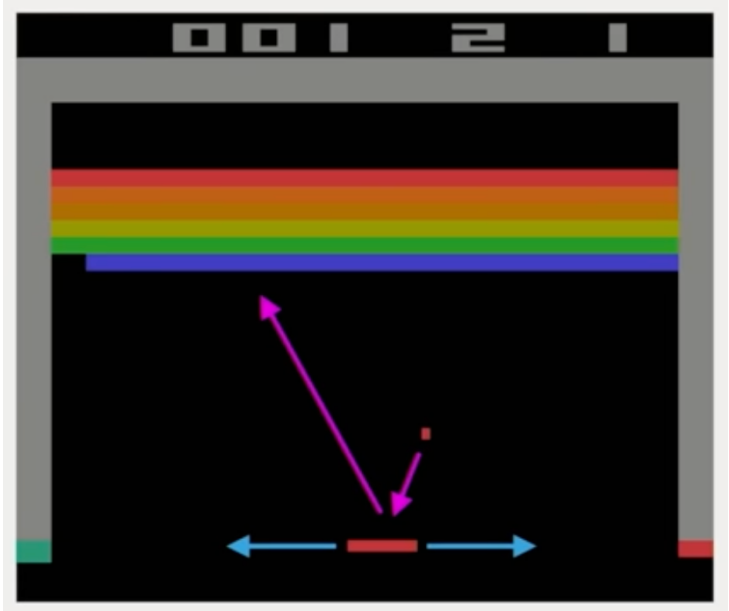
$$q_\pi(s,a) = \sum_{s',r} p(s', r \,|\, s, a)(r + v_\pi(s'))$$

Q-Learning

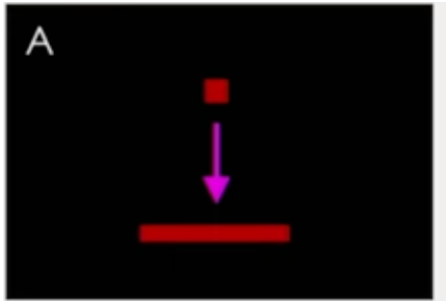
e.g.,

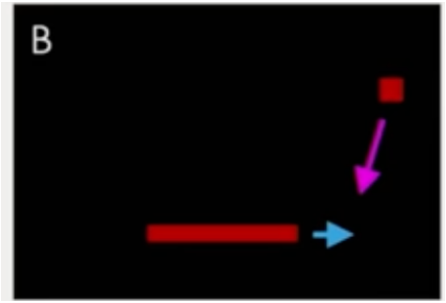
Estimate the optimal action-value function $Q^*(s,a)$

Which (s, a) pair has a higher Q-value?



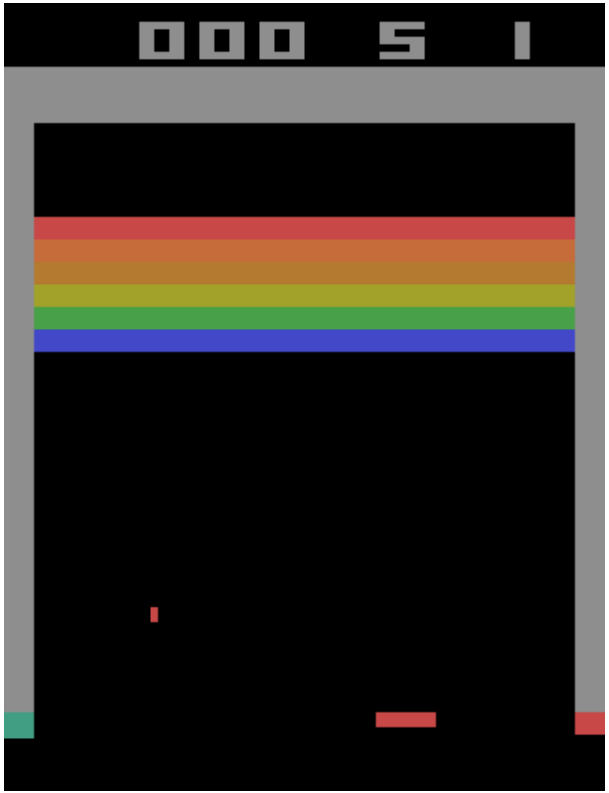
Atari Breakout game

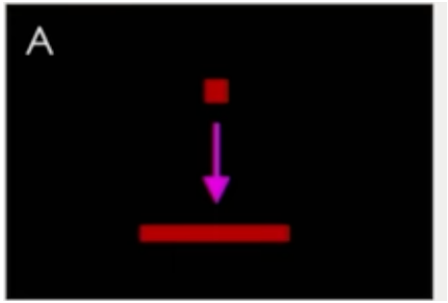


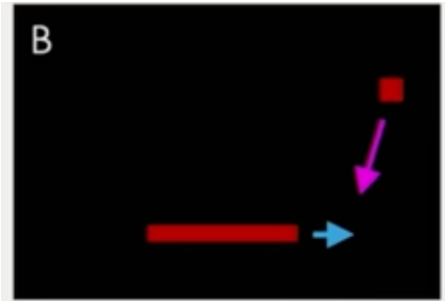


https://www.youtube.com/watch?v=TmPfTpjtdgg

Which (s, a) pair has a higher Q-value?







Value-based RL

Optimal Q-function is the maximum achievable value

$$Q^*(s,a) = \max_\pi Q^\pi(s,a) = Q^{\pi^*}(s,a)$$

Once we have it, we can act optimally

$$\pi^*(s) = \operatorname*{argmax}_a Q^*(s,a)$$

Optimal value maximizes over all future decisions

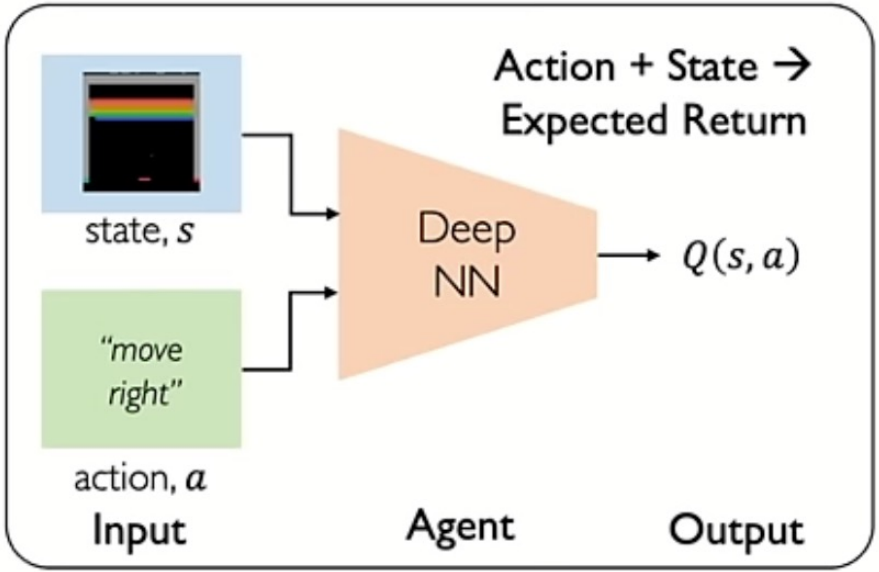
$$\begin{aligned} Q^*(s,a) &= r_{t+1} + \gamma \max_{a_{t+1}} r_{t+2} + \gamma^2 \max_{a_{t+2}} r_{t+3} + \dots \\ &= r_{t+1} + \gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1}) \end{aligned}$$

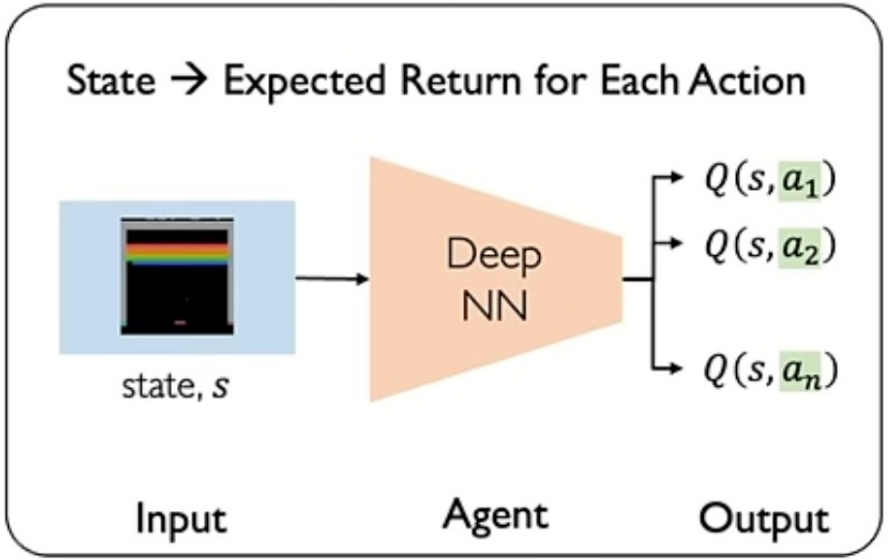
use a function approximator
to estimate Q(s,a)
e.g., a neural network

Formally, Q* satisfied Bellman Equations

$$Q^*(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^*(s',a') \mid s,a\right]$$

Deep Q-Network

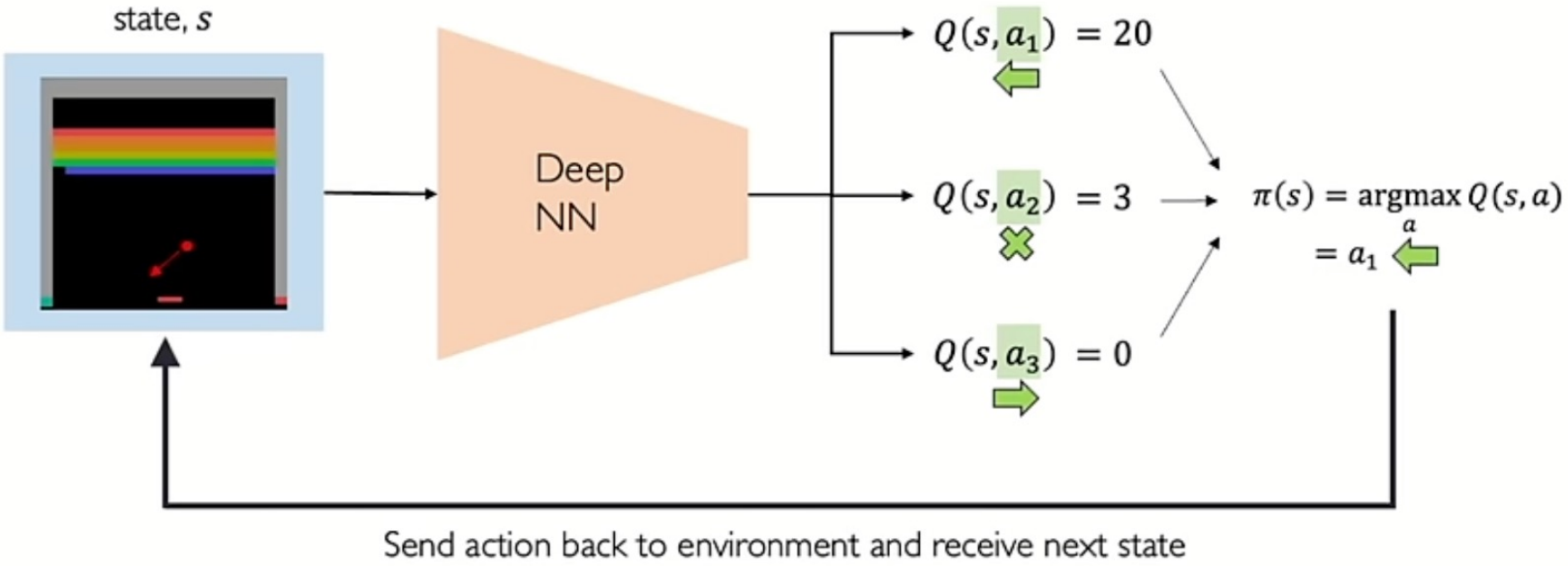




$$\mathcal{L} = \mathbb{E}\left[\left\|\overbrace{\left(r + \gamma \max_{a'} Q(s', a')\right)}^{\text{target}} - \overbrace{Q(s, a)}^{\text{predicted}}\right\|^2\right] \quad \textbf{Q-Loss}$$

* slide from Alexander Amini

Use NN to learn Q-function and then use to infer the optimal policy, $\pi(s)$



* slide from Alexander Amini

Actor-Critic Method

Problem: we don't know Q and V. Can we learn them?

Yes, using Q-learning! We can combine Policy Gradients and Q-learning by training both an **actor** (the policy) and a **critic** (the Q-function).

- The actor decides which action to take, and the critic tells the actor how good its action was and how it should adjust
- Also alleviates the task of the critic as it only has to learn the values of (state, action) pairs generated by the policy
- Can also incorporate Q-learning tricks e.g. experience replay
- **Remark:** we can define by the **advantage function** how much an action was better than expected

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

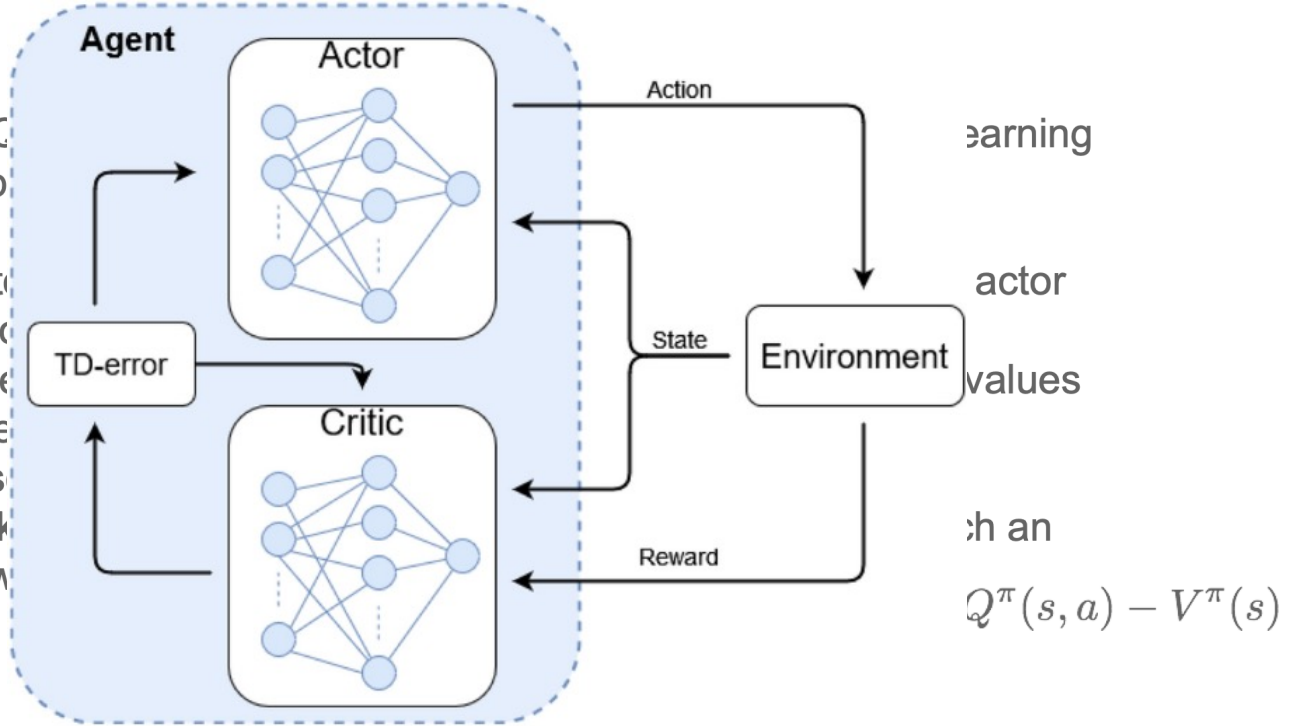
Actor-Critic Method

Problem: we don't know Q and V. Can we learn them?

Yes, using C earning
by training b

- The act actor
  how goo
- Also alle values
  of (state
- Can als
- **Remark** ch an
  action w

$Q^\pi(s,a) - V^\pi(s)$



Policy gradients: very general but suffer from high variance so requires a lot of samples. ***Challenge***: sample-efficiency

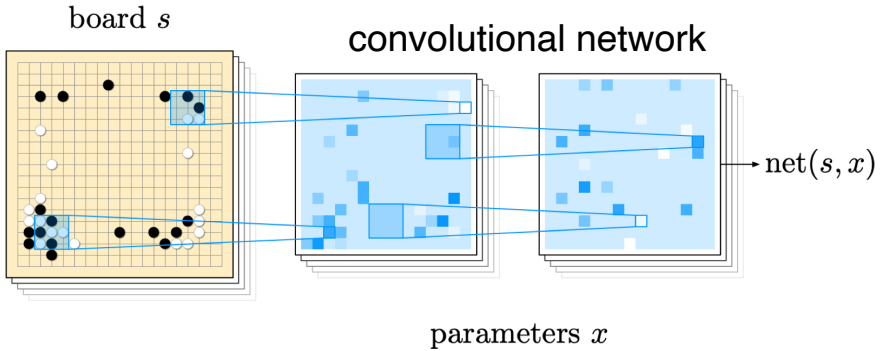
Q-learning: does not always work but when it works, usually more sample-efficient. ***Challenge***: exploration

Guarantees:

— Policy Gradients: Converges to a local minima of $J(\theta)$, often good enough!

— Q-learning: Zero guarantees since you are approximating Bellman equation with a complicated function approximator

AlphaGo



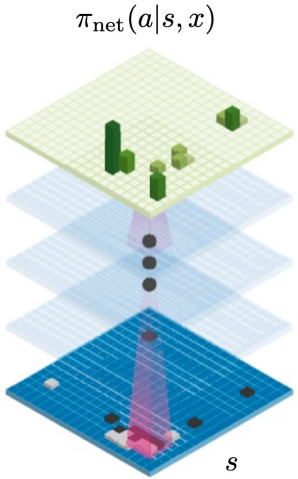
Policy network

- **Dataset**: KGS server games split into board / next-move pairs (s_i, a_i)
  - 160,000 games $\rightarrow$ 29 million (s_i, a_i) pairs.
- **Loss:** negative log-likelihood,

$$-\sum_{i=1}^{N} \log \pi_{\mathrm{net}}(a_i|s_i, x).$$

- Use trained net as a Go player:

$$a^* = \arg\max_a \{\log \pi_{\mathrm{net}}(a|s, x)\}.$$



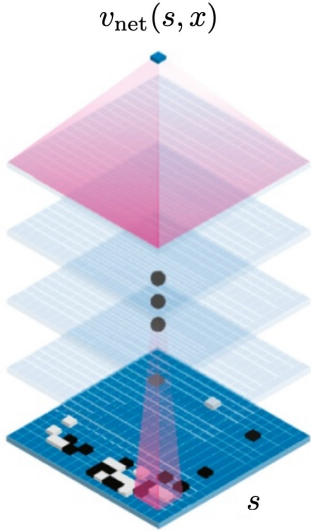
(Silver et al., 2016)

Value network

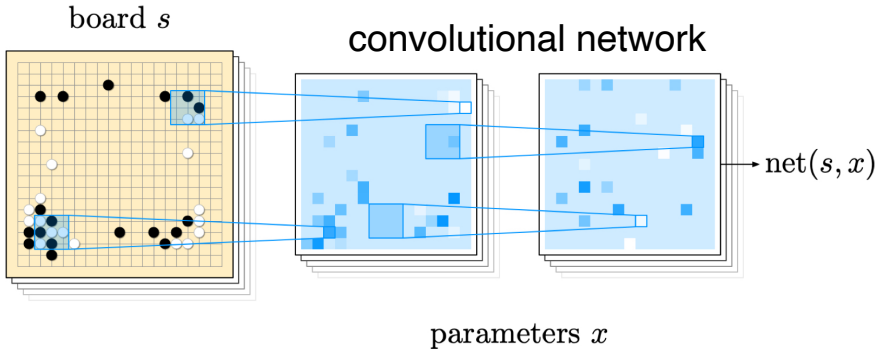
- **Dataset**: KGS server games split into board / outcome pairs $(s_i, o(s_i))$
- **Loss:** squared error,

$$\sum_{i=1}^{N} (o(s_i) - v_{\mathrm{net}}(s_i, x))^2.$$

- **Problem**: Effective sample size of 160,000 games was not enough.



AlphaGo



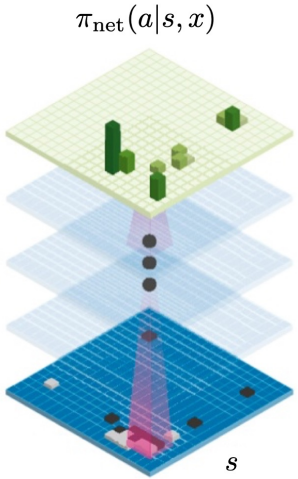
Policy network

- **Dataset**: KGS server games split into board / next-move pairs (s_i, a_i)
  - 160,000 games $\rightarrow$ 29 million (s_i, a_i) pairs.
- **Loss:** negative log-likelihood,

$$-\sum_{i=1}^{N} \log \pi_{\mathrm{net}}(a_i|s_i, x).$$

- Use trained net as a Go player:

$$a^* = \arg\max_a \{\log \pi_{\mathrm{net}}(a|s, x)\}.$$



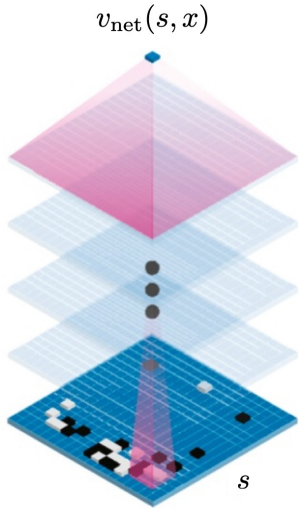
(Silver et al., 2016)

Value network

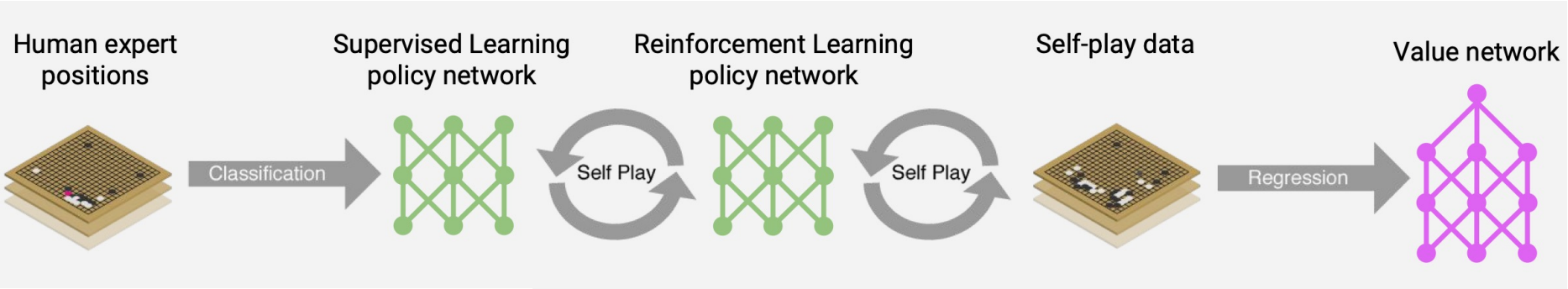
Successful attempt.

- Use Policy Net playing against itself to **generate millions of unique games**.
- **Dataset**: Board / outcome pairs $(s_i, o(s_i))$, each from a unique self-play game.
- **Loss:** squared error,

$$\sum_{i=1}^{N} (o(s_i) - v_{\mathrm{net}}(s_i, x))^2.$$



AlphaGo

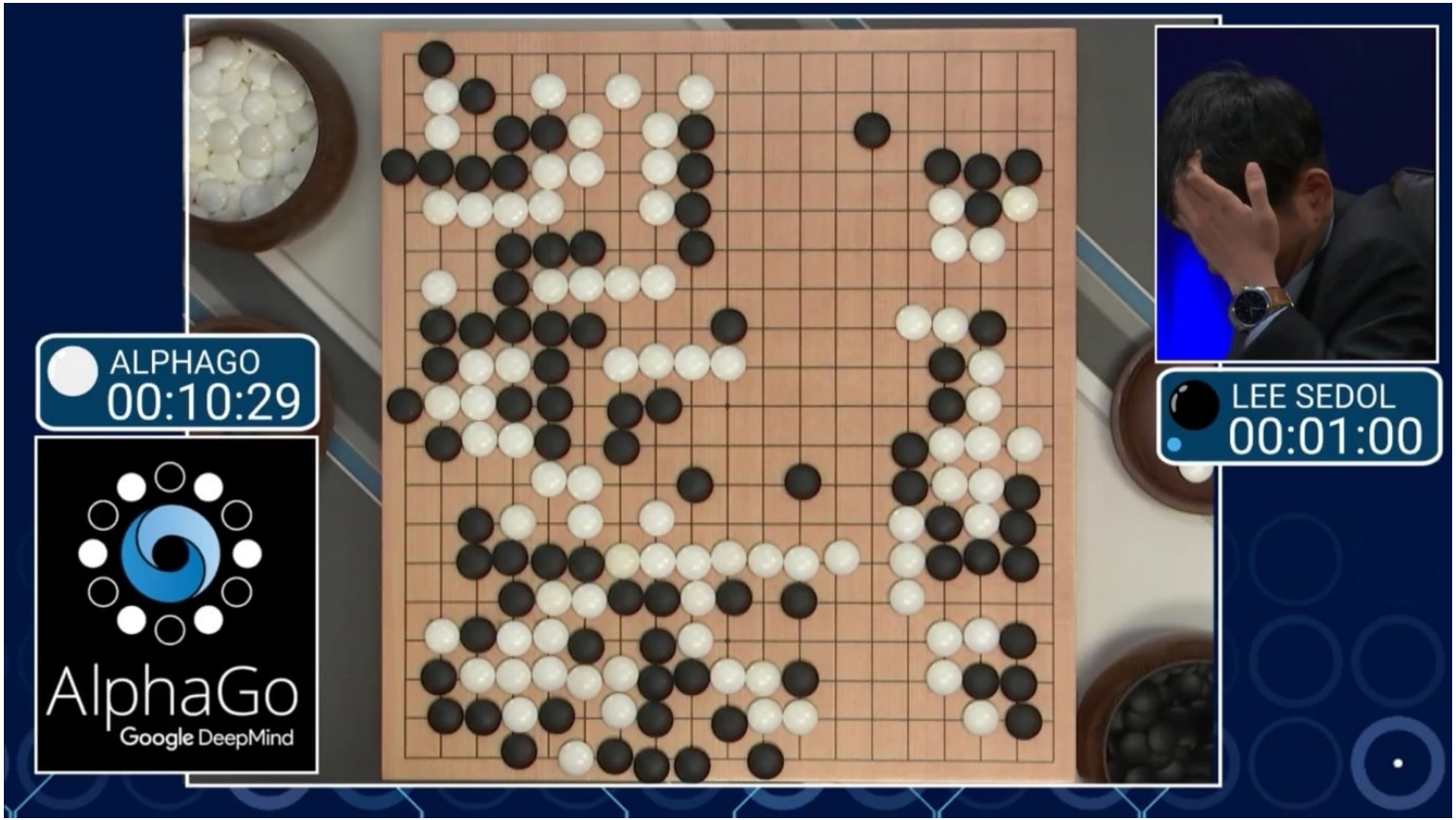


1) Initial training: human data

2) Self-play and reinforcement learning
→ super-human performance

3) "Intuition" about board state

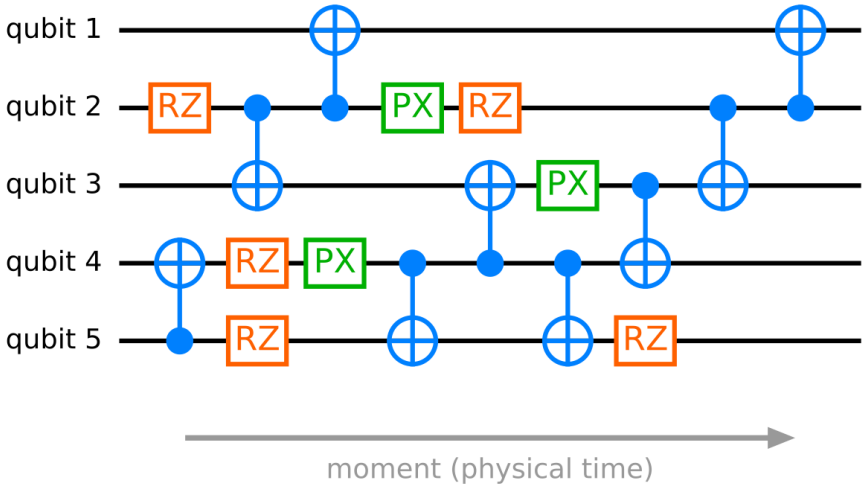
AlphaGo



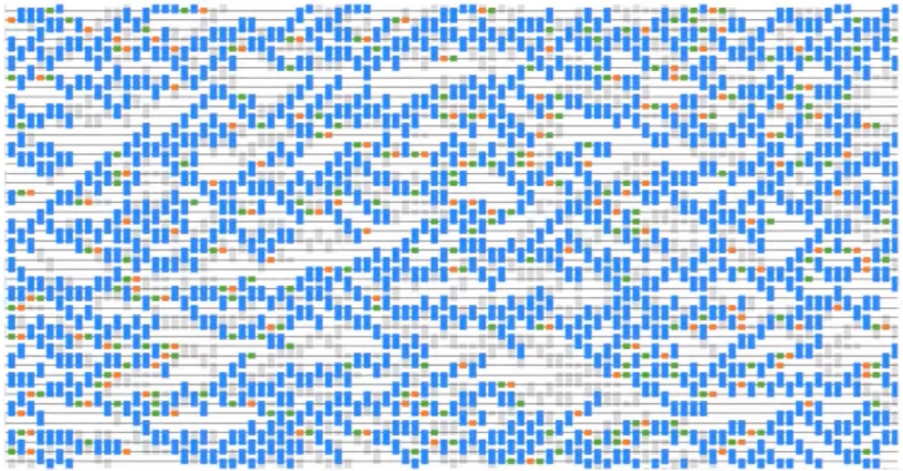
Applications in quantum physics

Quantum circuit optimization with RL

https://arxiv.org/abs/2103.07585

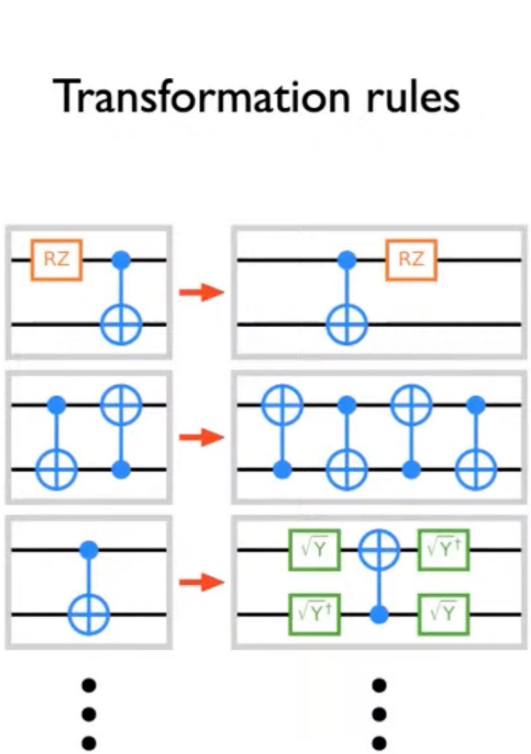


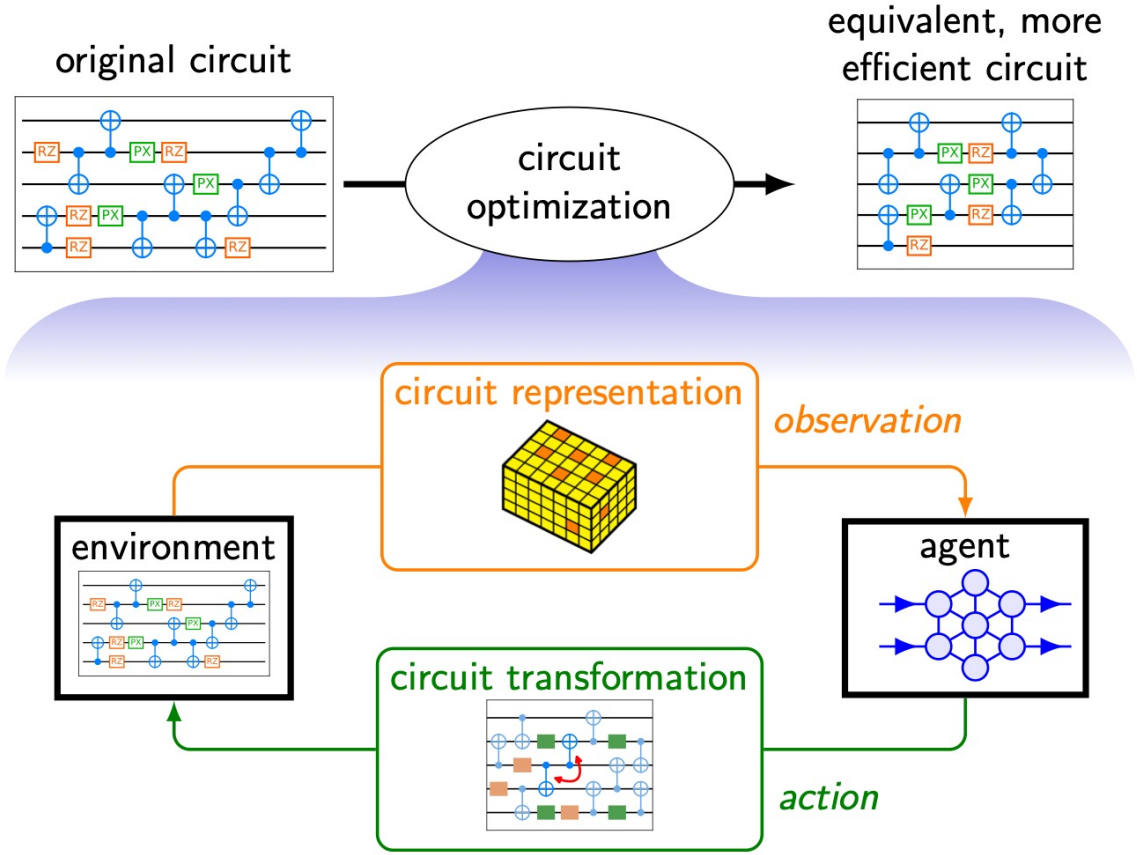
NISQ devices: Quantum Circuit Optimization critical
…and needs to be hardware-dependent
[not on an abstract level designed for large-scale fault-tolerant circuits]



Quantum circuit optimization with RL

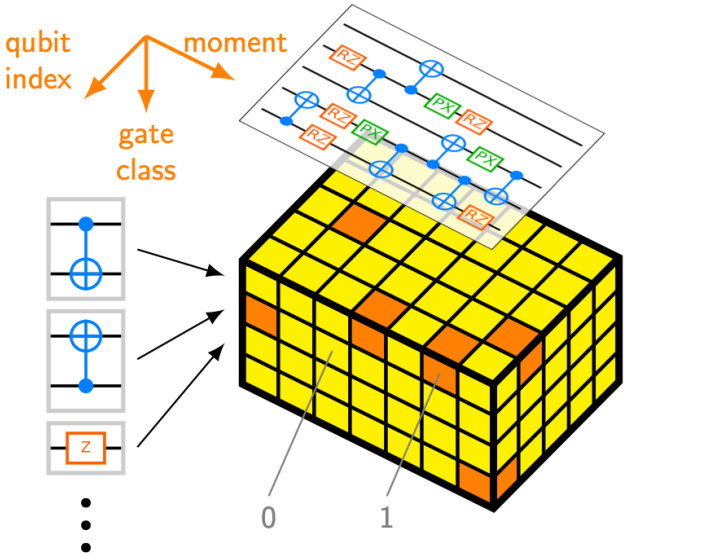
https://arxiv.org/abs/2103.07585





Quantum circuit optimization with RL

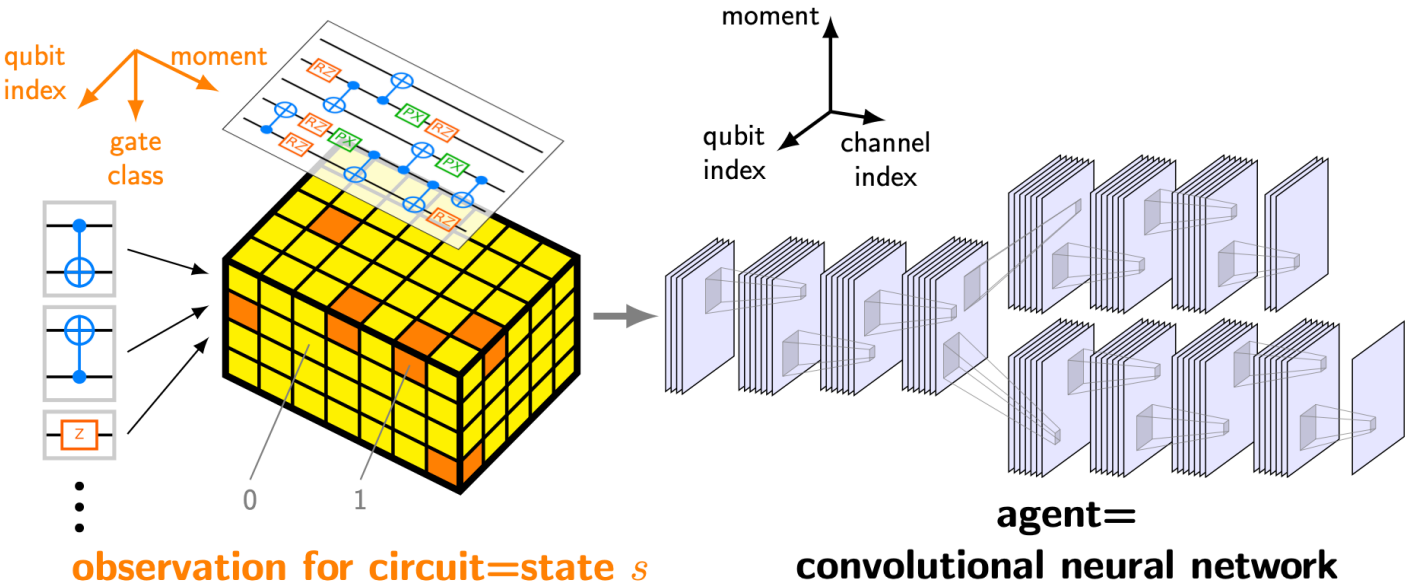
https://arxiv.org/abs/2103.07585



observation for circuit=state s

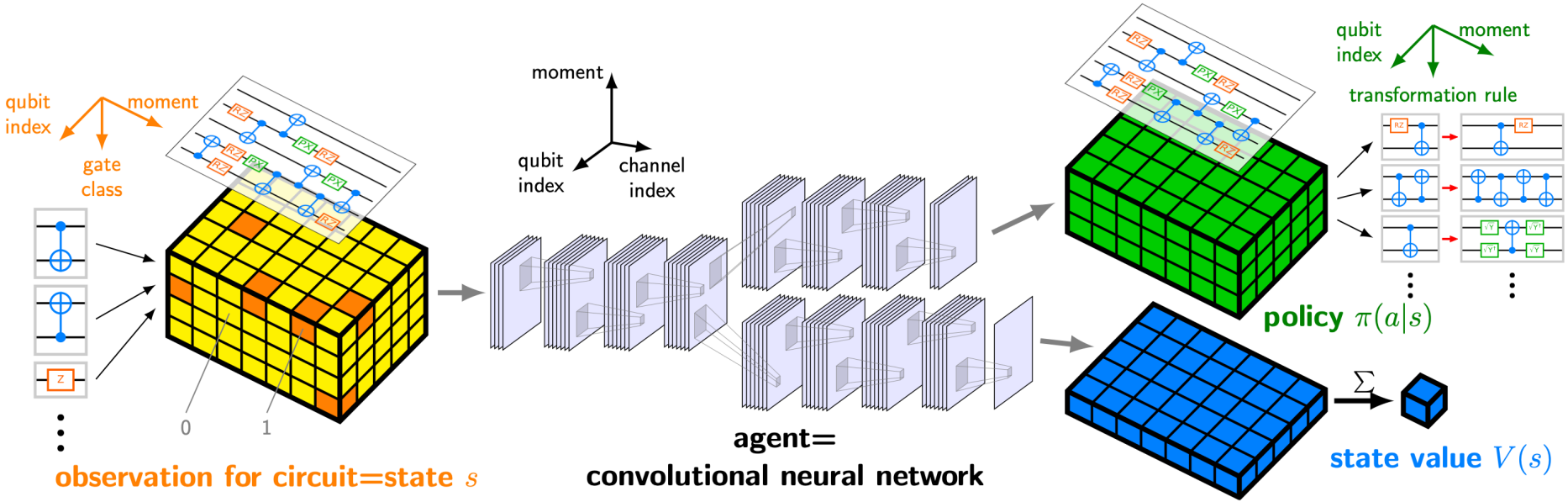
Quantum circuit optimization with RL

https://arxiv.org/abs/2103.07585



Quantum circuit optimization with RL

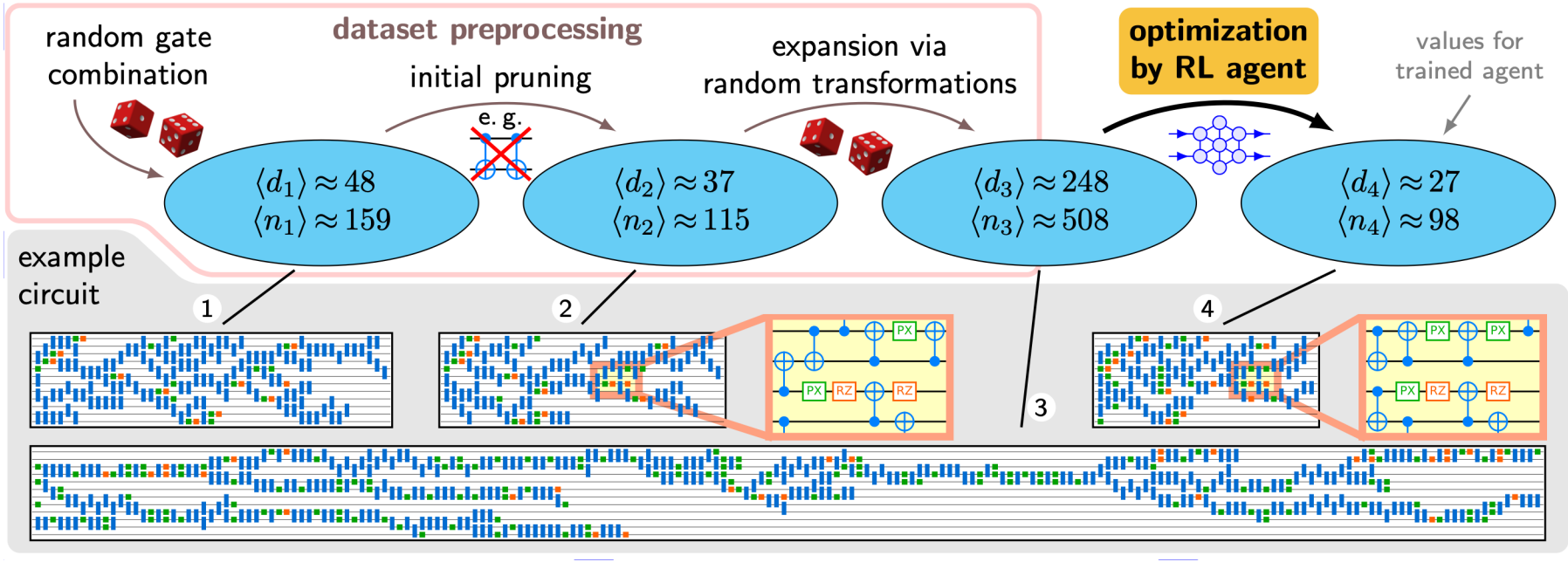
https://arxiv.org/abs/2103.07585



Reward: reduction in gate count, depth, or combination
(possibly: gate-dependent, decoherence estimate, …)

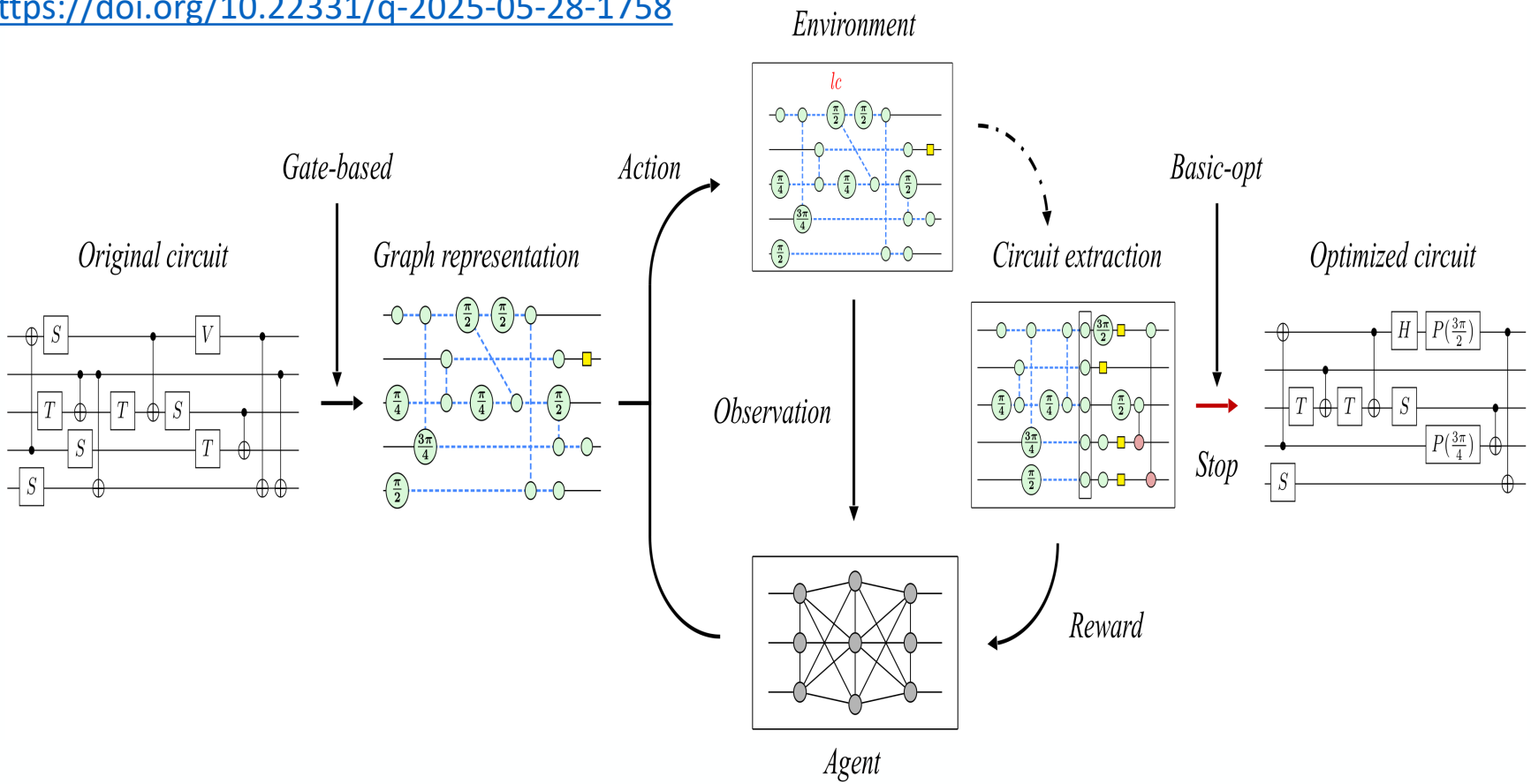
Quantum circuit optimization with RL

https://arxiv.org/abs/2103.07585



Quantum circuit optimization with RL

https://doi.org/10.22331/q-2025-05-28-1758

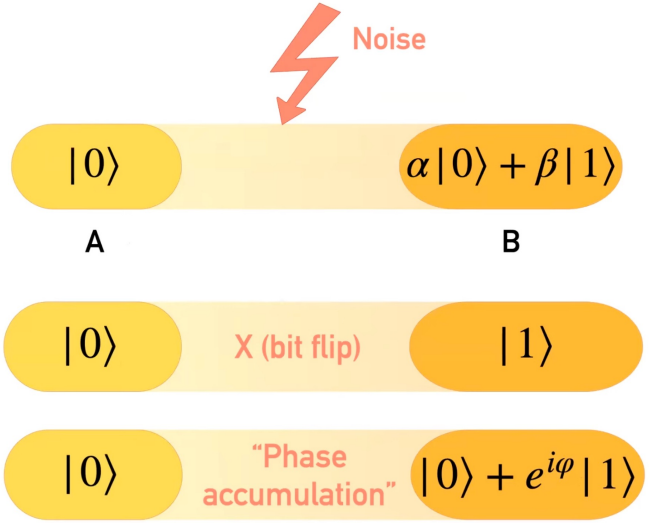


Quantum error correction

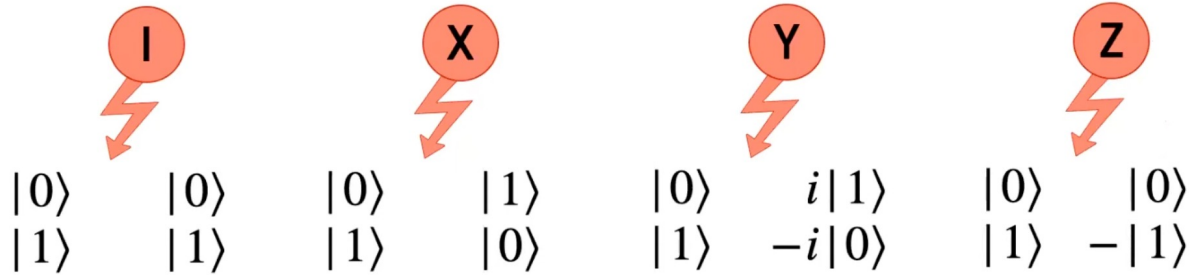
Redundancy protects qubits



Encode logical qubits using physical qubits

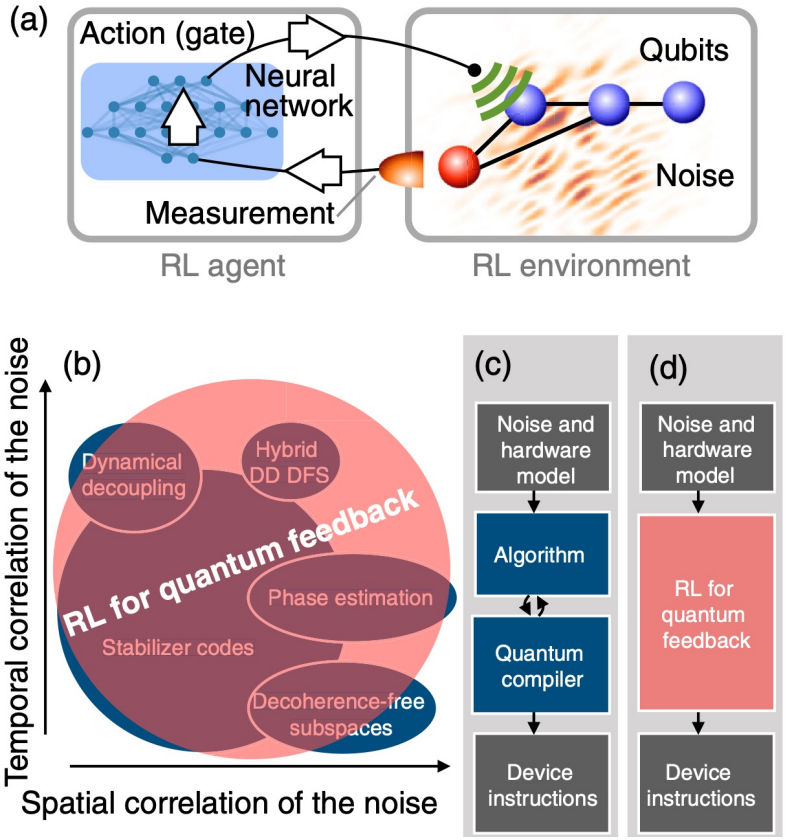


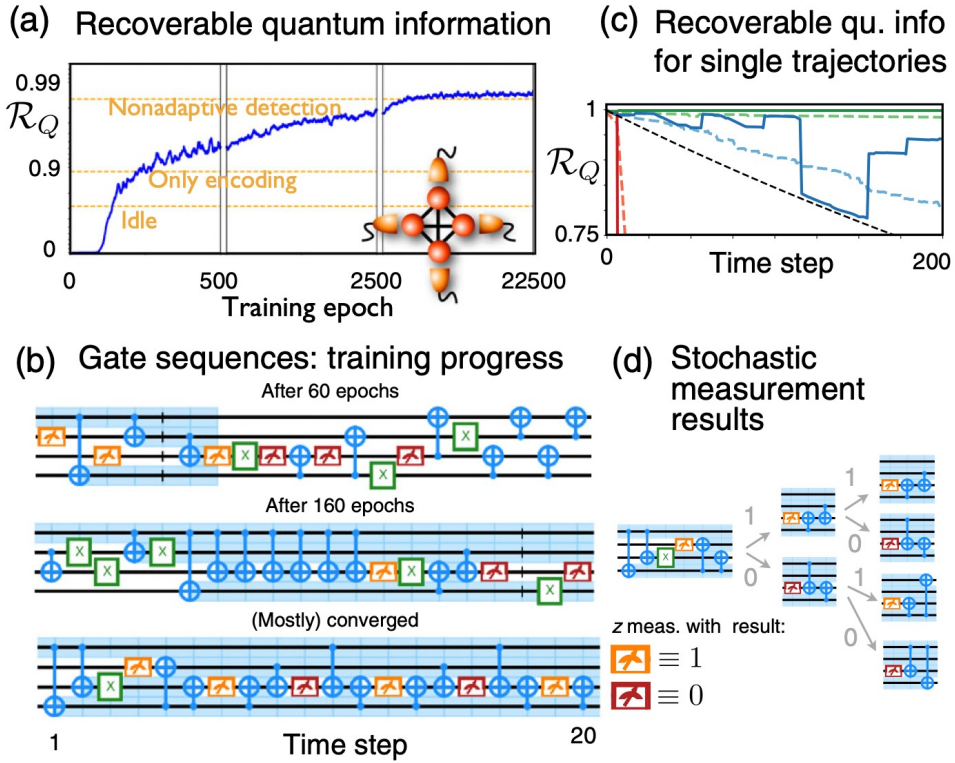
Any error $\mathscr{E}$ = Linear combination of I, X, Y, Z



Quantum error correction with RL

https://doi.org/10.1103/PhysRevX.8.031084

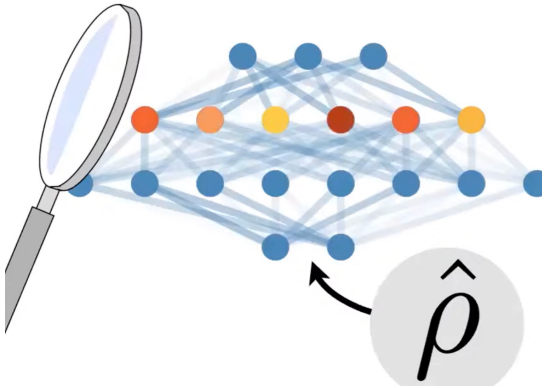


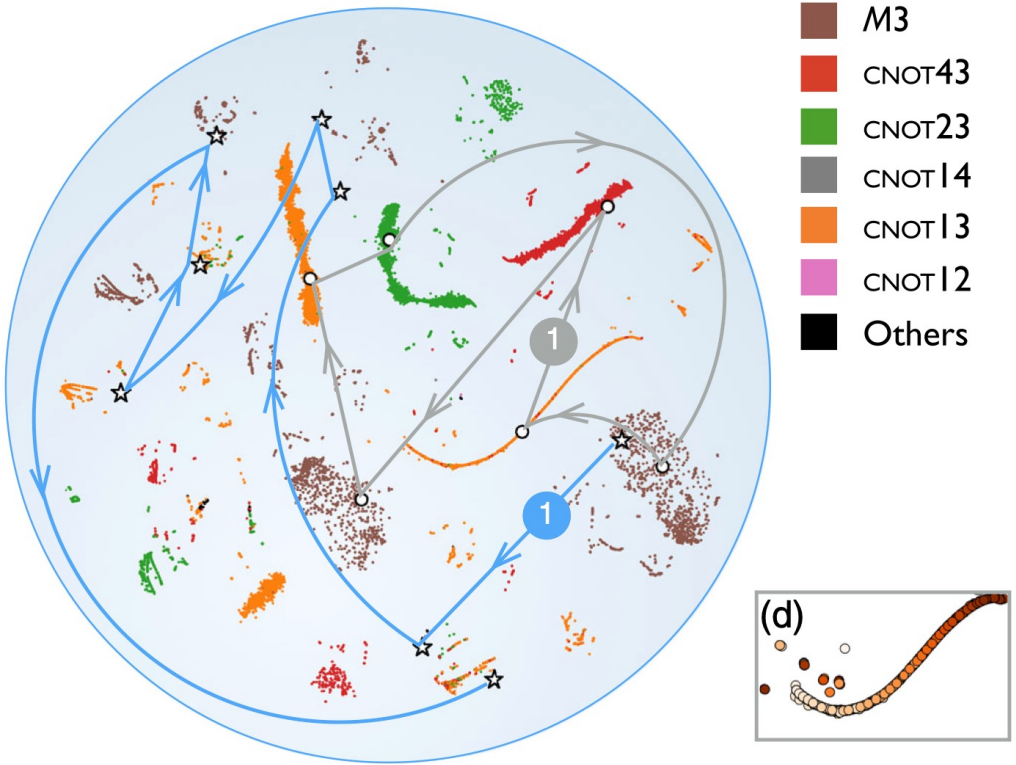


Quantum error correction with RL

https://doi.org/10.1103/PhysRevX.8.031084

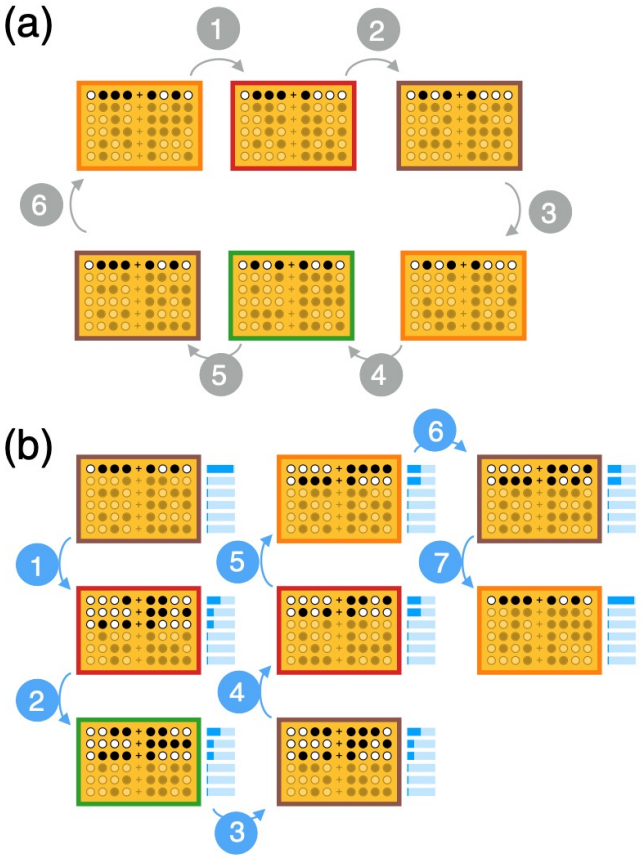
How does the network operate?

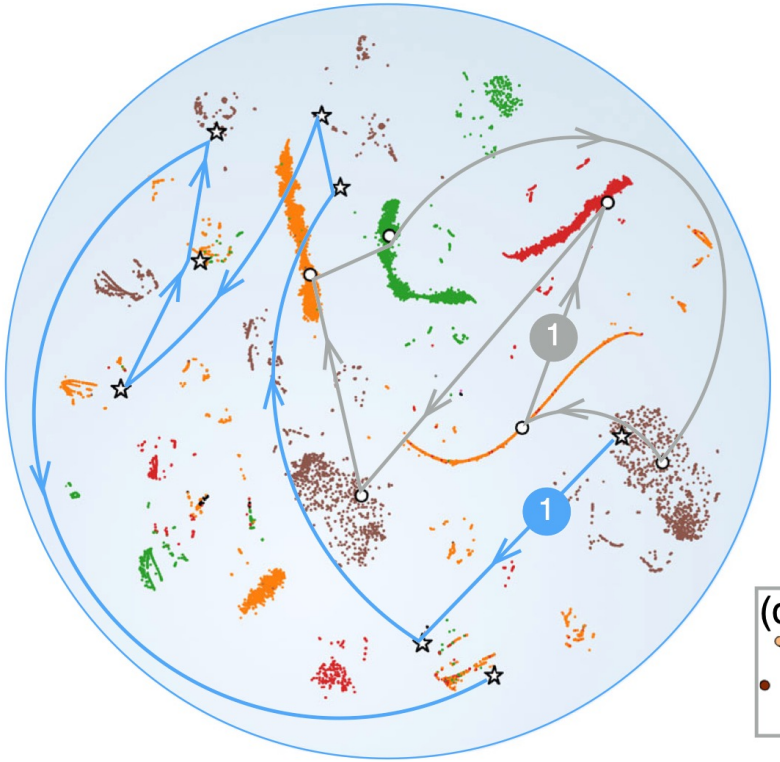


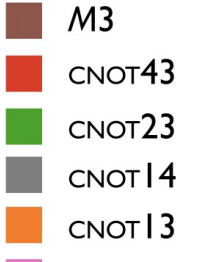


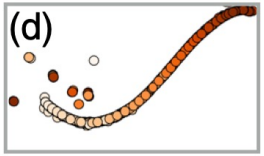
Quantum error correction with RL

https://doi.org/10.1103/PhysRevX.8.031084



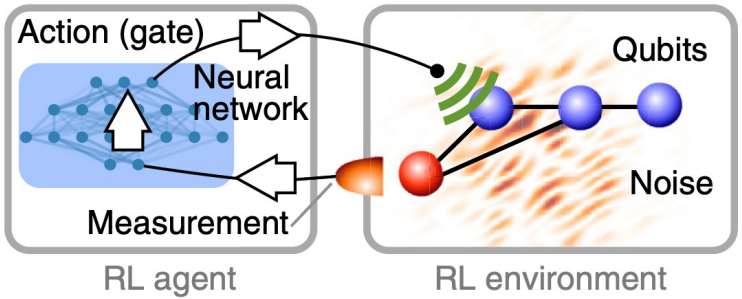


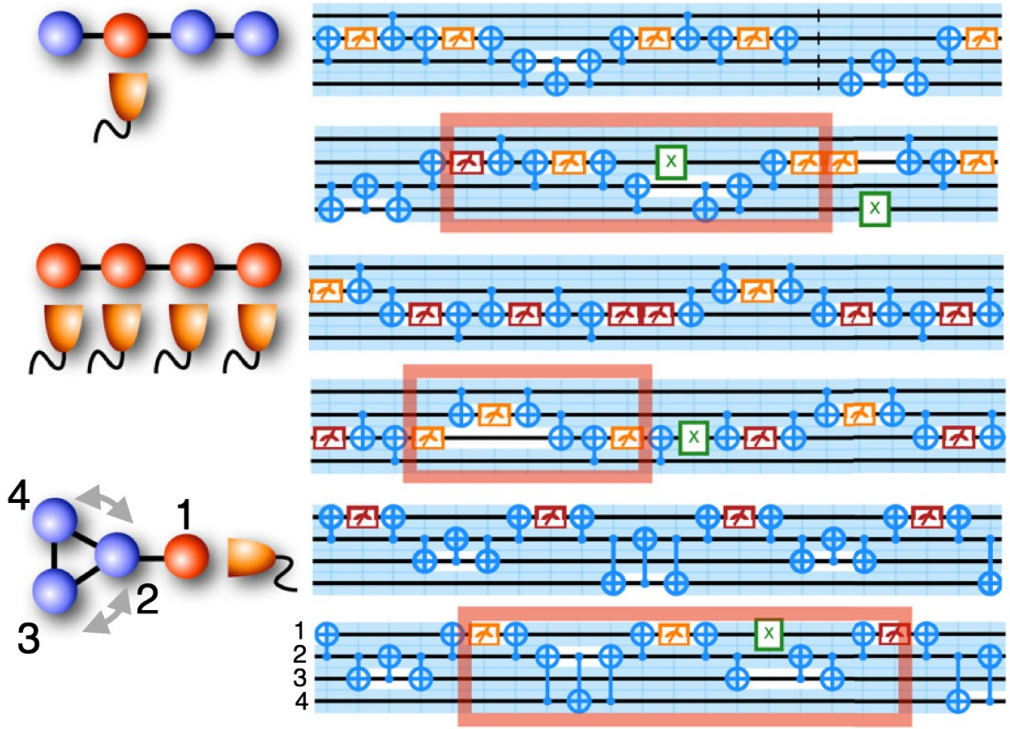




Quantum error correction with RL

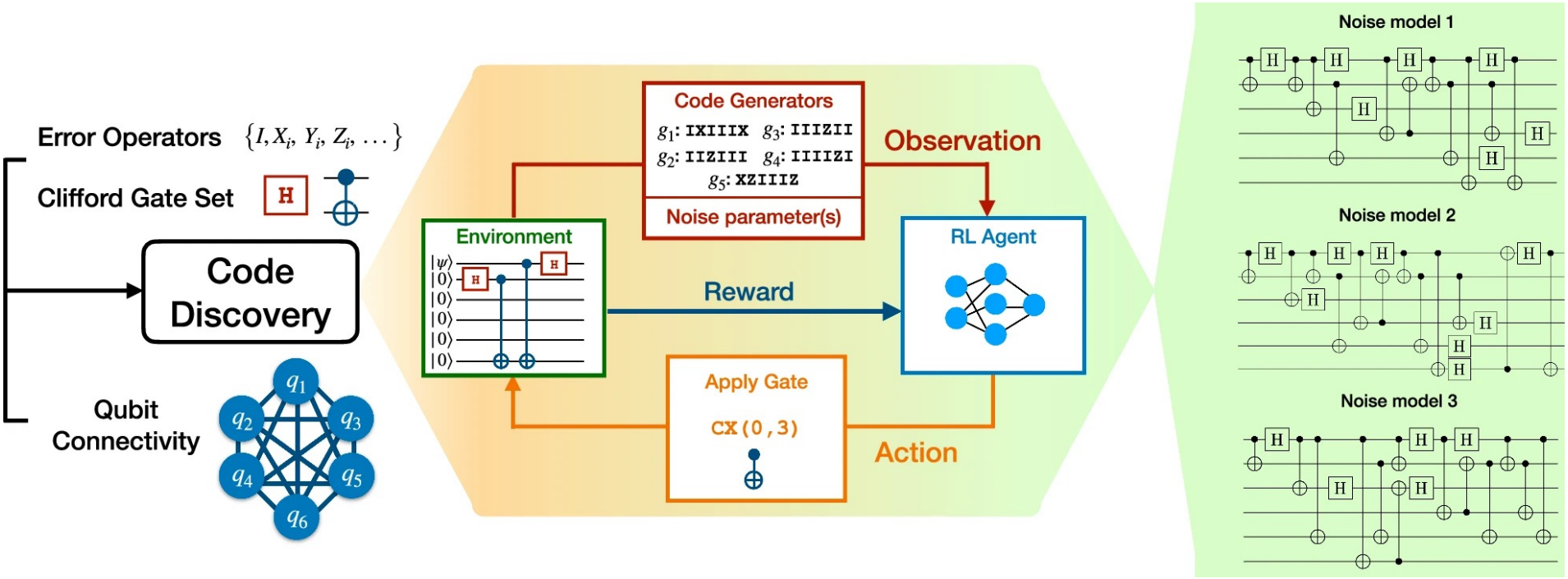
https://doi.org/10.1103/PhysRevX.8.031084





Quantum error correction with RL

https://doi.org/10.1038/s41534-024-00920-y



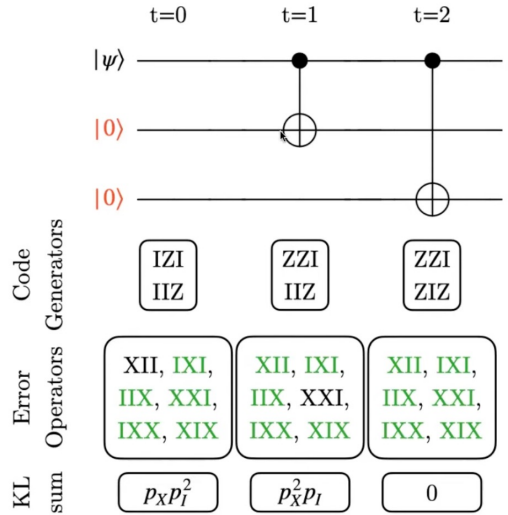
Quantum error correction with RL

https://doi.org/10.1038/s41534-024-00920-y

Task: correct all single-qubit bit flips

↓

Detect all single and two-qubit bit flips



Error operators: $\{E_\mu\}$

Associated probabilities: $\{p_\mu\}$

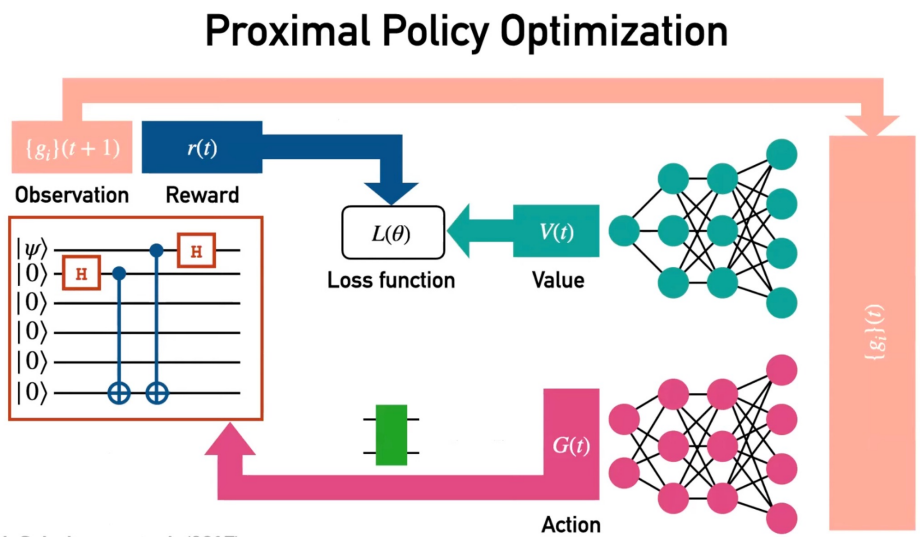
$$r_t = -\sum_\mu p_\mu K_\mu$$

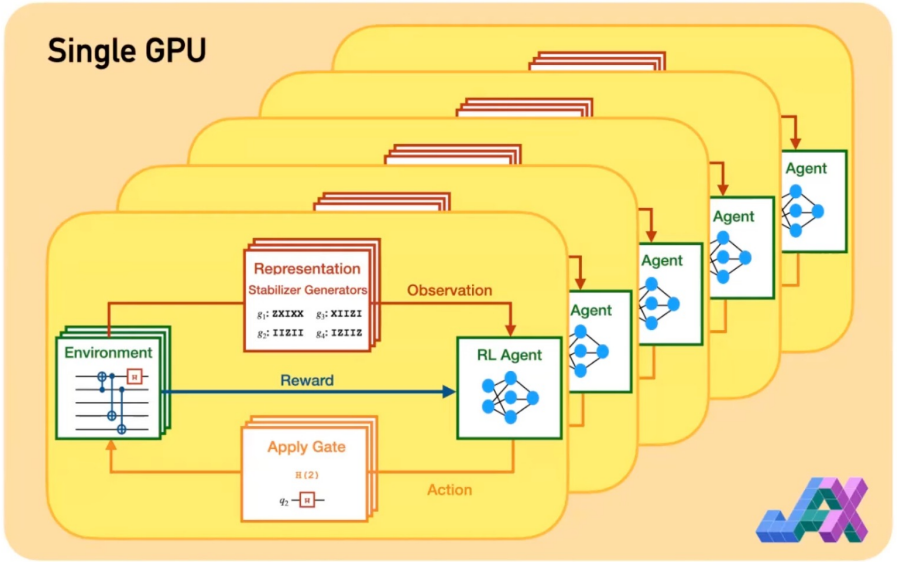
$K_\mu = 0$ if $\{E_\mu, g_i\} = 0$, for some g_i, or $E_\mu \in S_C$

$K_\mu = 1$ otherwise

Quantum error correction with RL

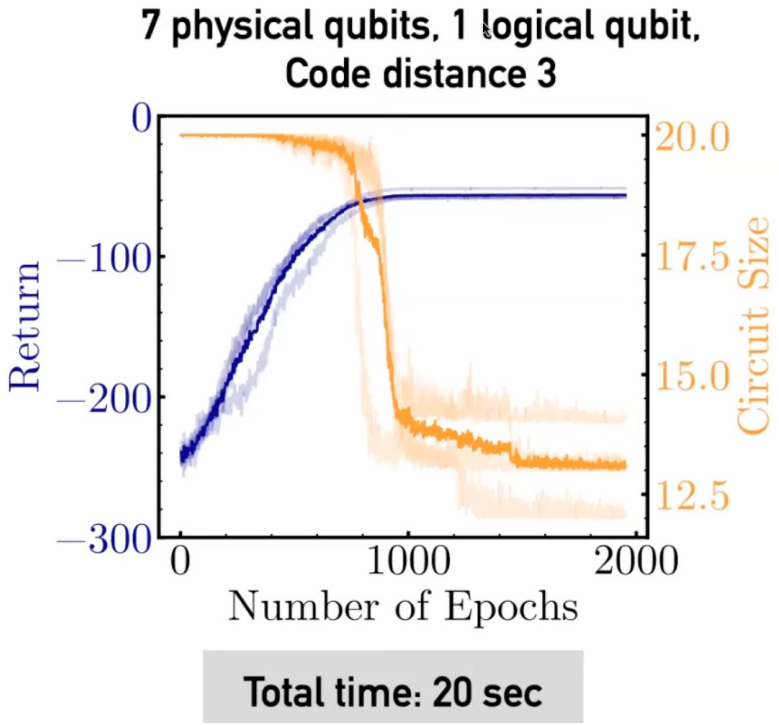
https://doi.org/10.1038/s41534-024-00920-y

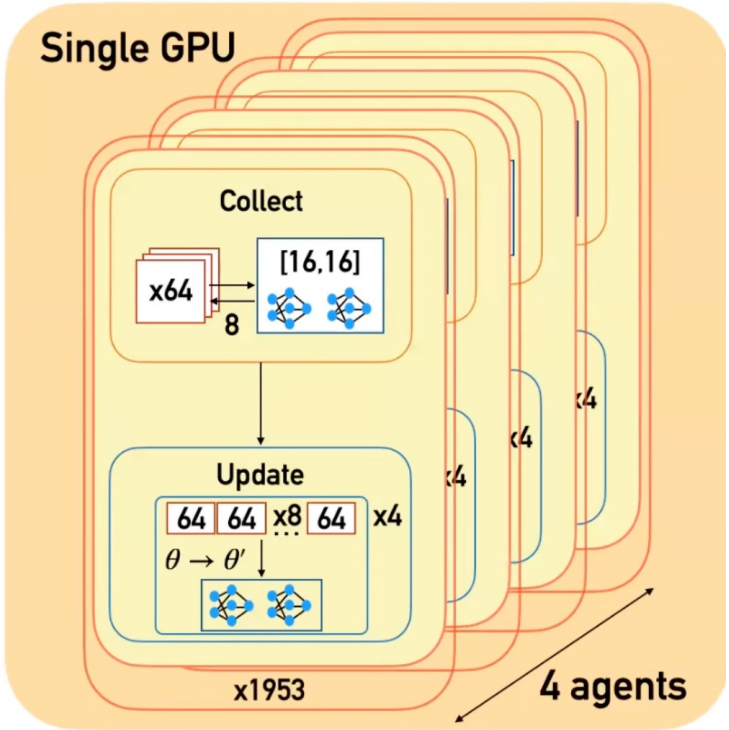




Quantum error correction with RL

https://doi.org/10.1038/s41534-024-00920-y

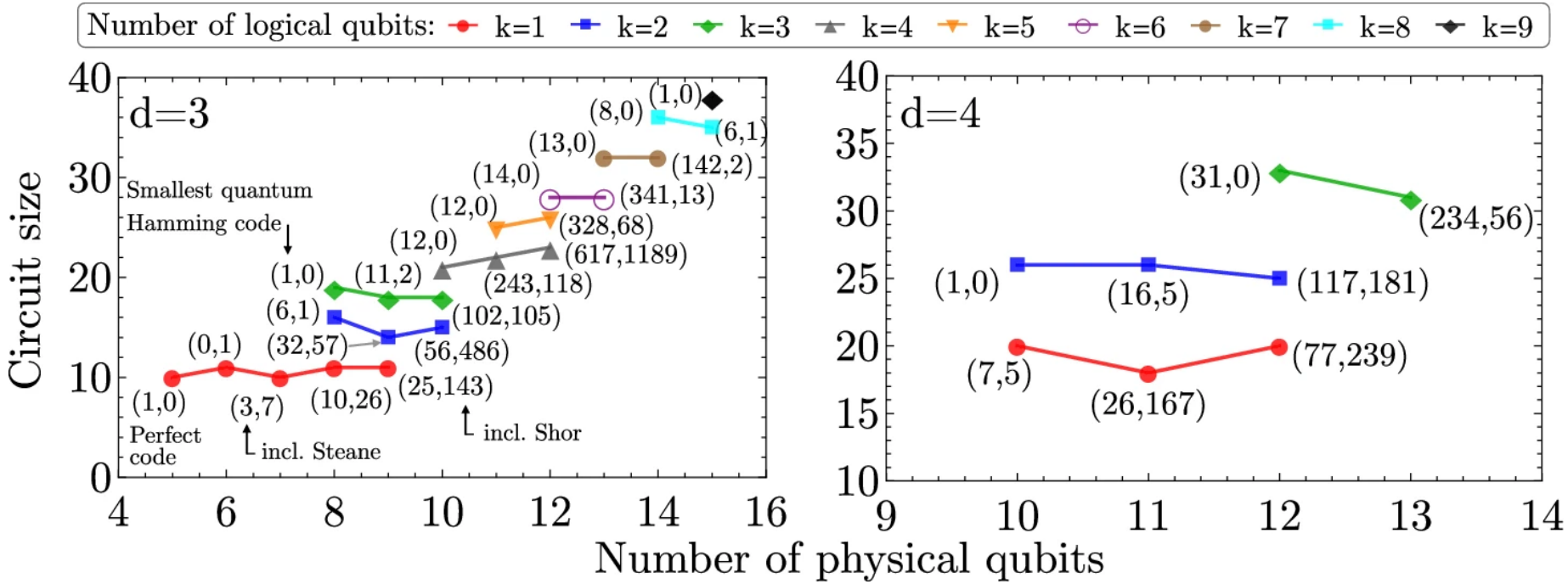




Quantum error correction with RL

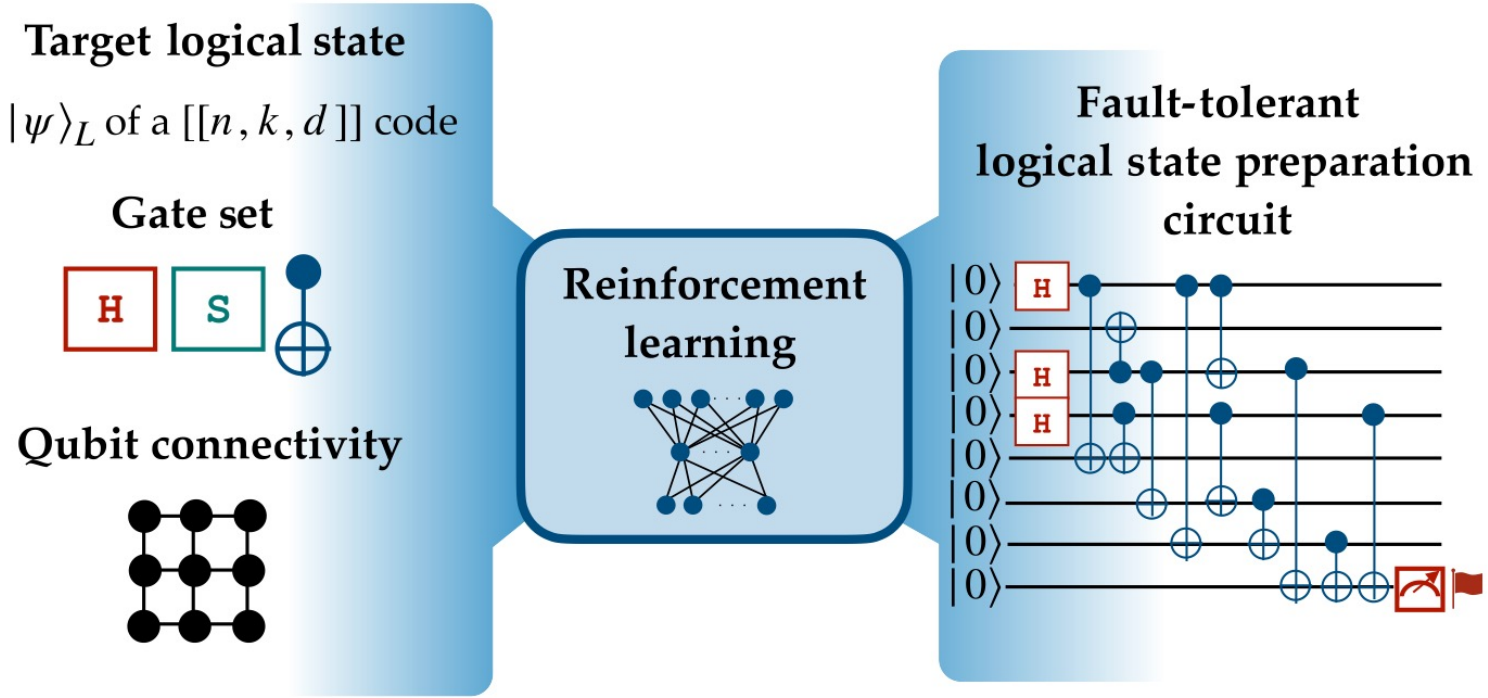
https://doi.org/10.1038/s41534-024-00920-y

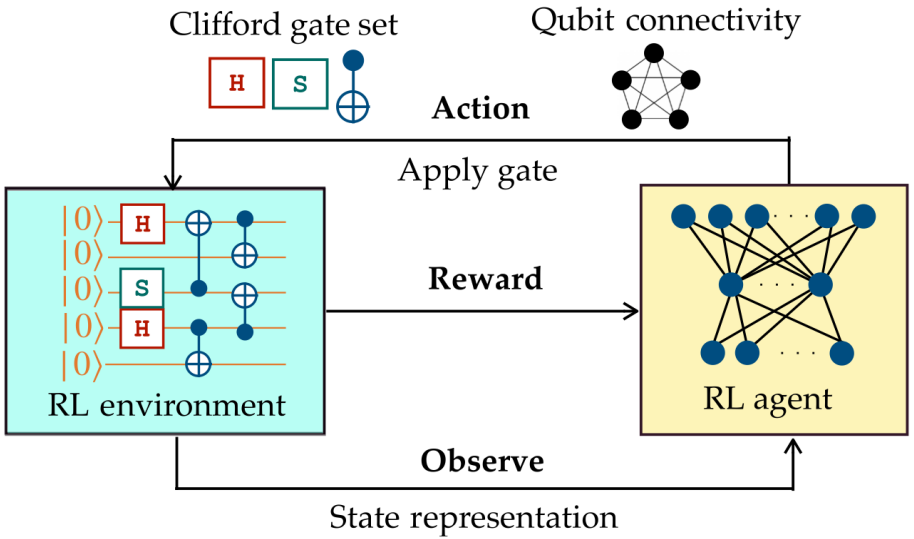
stabilizer codes



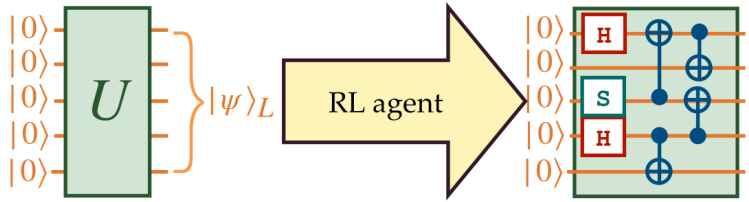
Discovery of fault-tolerant logical state preparation circuit

https://doi.org/10.1103/gqpr-dgz7

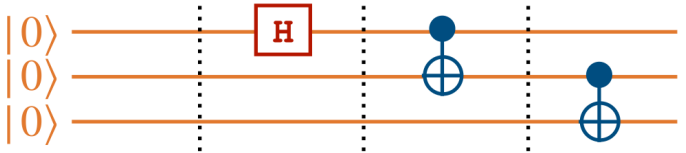




(a)

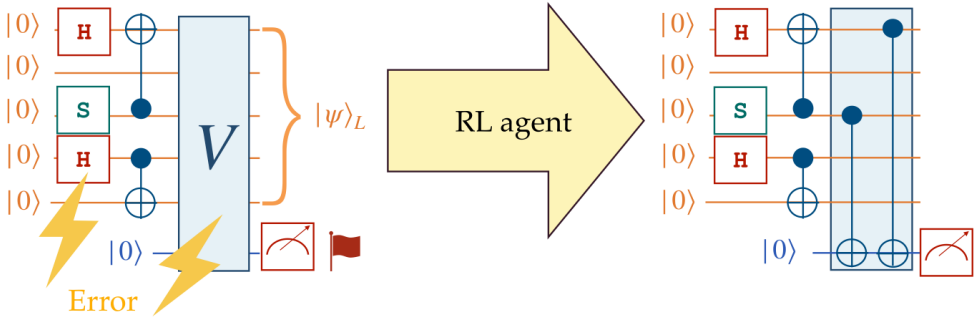


(b) Reward function

	$t=0$	$t=1$	$t=2$	$t=3$
Circuit				
Canonical tableau	g_1 : +ZII g_2 : +IZI g_3 : +IIZ	+XII +IZI +IIZ	+XXI +ZZI +IIZ	+XXX +ZIZ +IZZ
State $\lvert\psi_t\rangle$	$\lvert 000\rangle$	$\lvert 000\rangle+\lvert 100\rangle$	$\lvert 000\rangle+\lvert 110\rangle$	$\lvert 000\rangle+\lvert 111\rangle$
Fidelity $\lvert\langle\psi_t\lvert\psi_{\text{target}}\rangle\rvert^2$	0.5	0.25	0.25	1.0
Energy $-\langle\psi_t\lvert H\rvert\psi_t\rangle$	2	1	1	3
Compl. distance $1-d_t$	0.25	0.43	0.71	1.0

VERIFICATION CIRCUIT SYNTHESIS

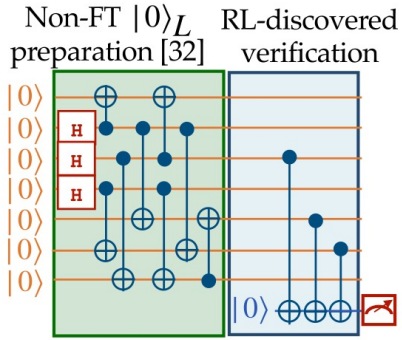
synthesize a circuit V and use the ancilla flag qubits to flag harmful errors and thereby render the encoding protocol fault tolerant



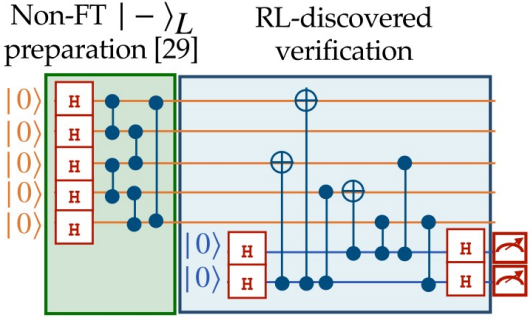
$$f_t = \sum_{E \in \mathcal{E}} \begin{cases} 0 & \text{if } E \text{ is } I \text{ and a flag is triggered} \\ 1 & \text{if } E \text{ is tolerable} \\ 1 & \text{if } E \text{ is harmful and a flag is triggered} \\ 0 & \text{if } E \text{ is harmful and flags are not triggered.} \end{cases}$$

$$r_t = \mu_f(f_t - f_{t-1}) + \mu_d(d_{t-1} - d_t) + \mu_p(p_t - p_{t-1}),$$

(a) [[7,1,3]] Steane



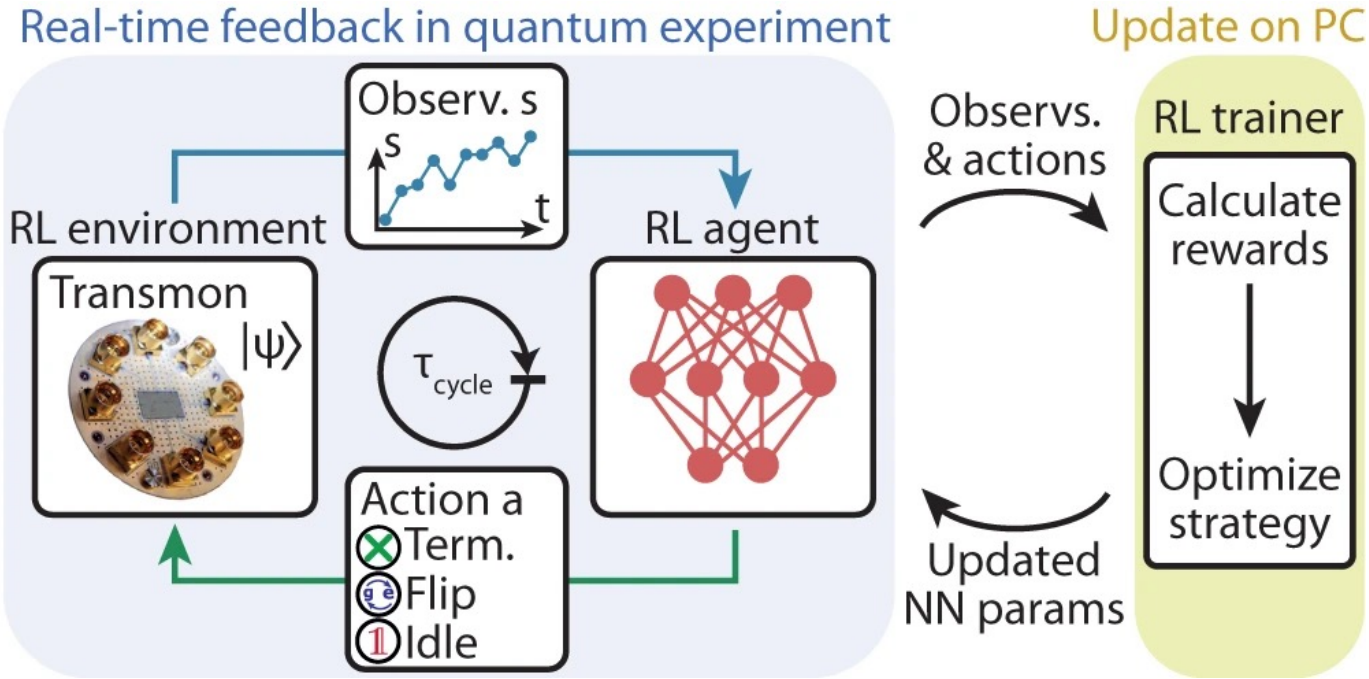
(b) [[5,1,3]] perfect



Quantum Control with RL

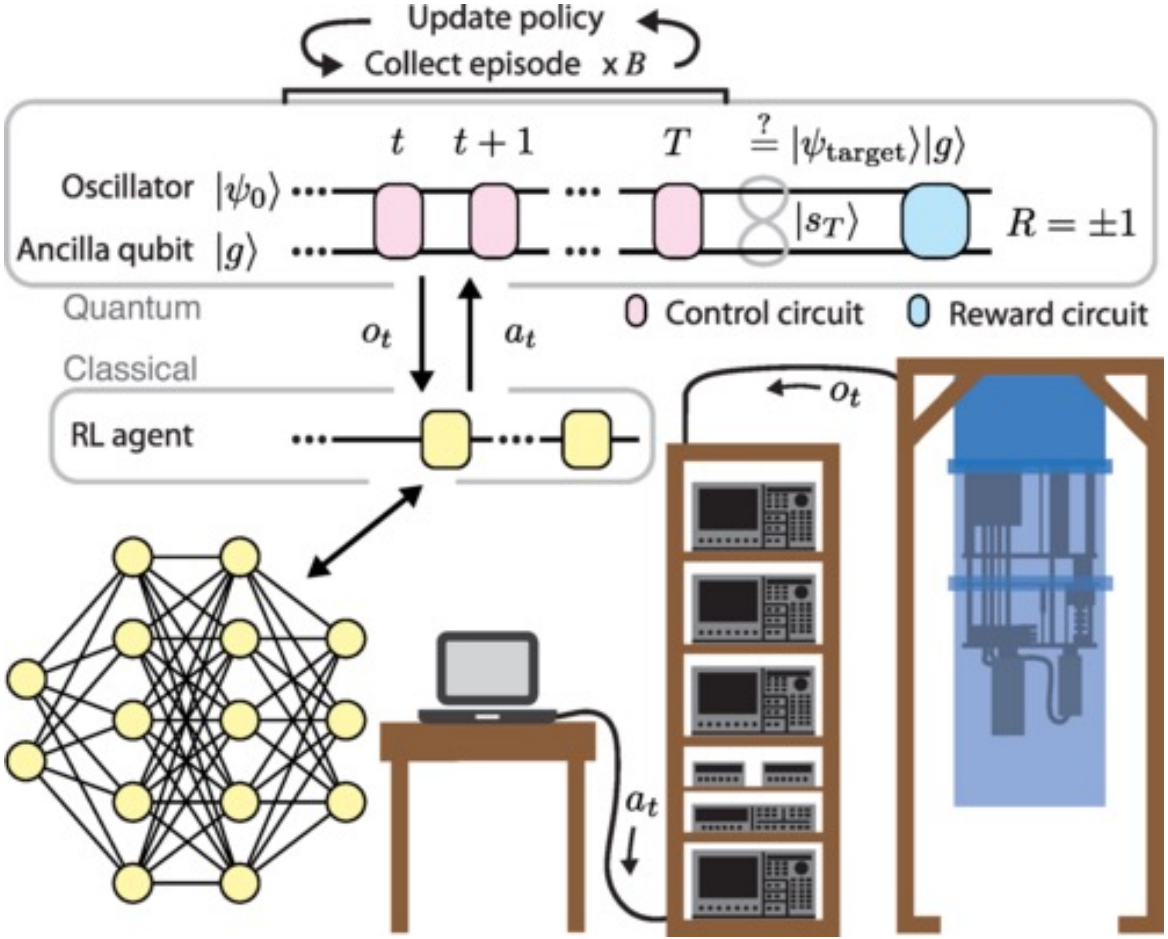
https://doi.org/10.1038/s41467-023-42901-3

observations are obtained via quantum measurements, actions are realized as unitary gate operations, and the reward is measured in terms of the speed and fidelity of initializing the quantum system into a target state



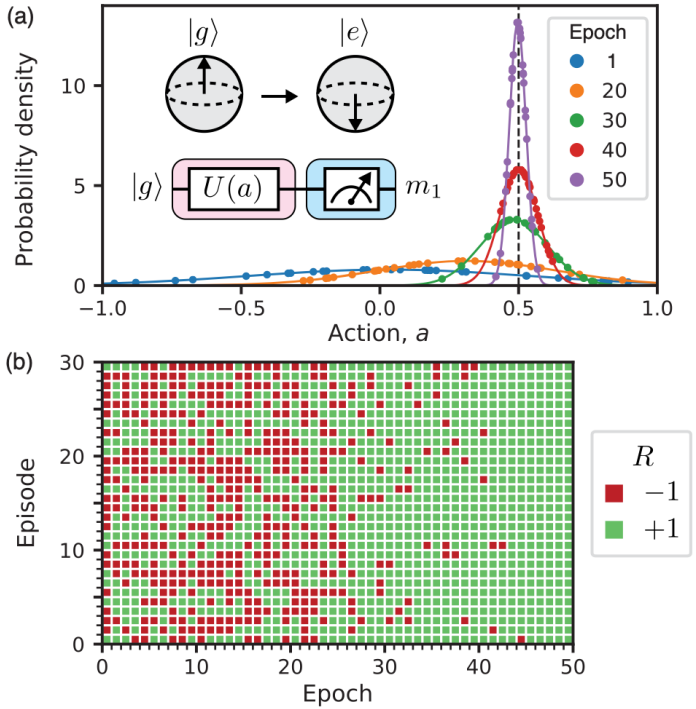
Quantum Control with RL

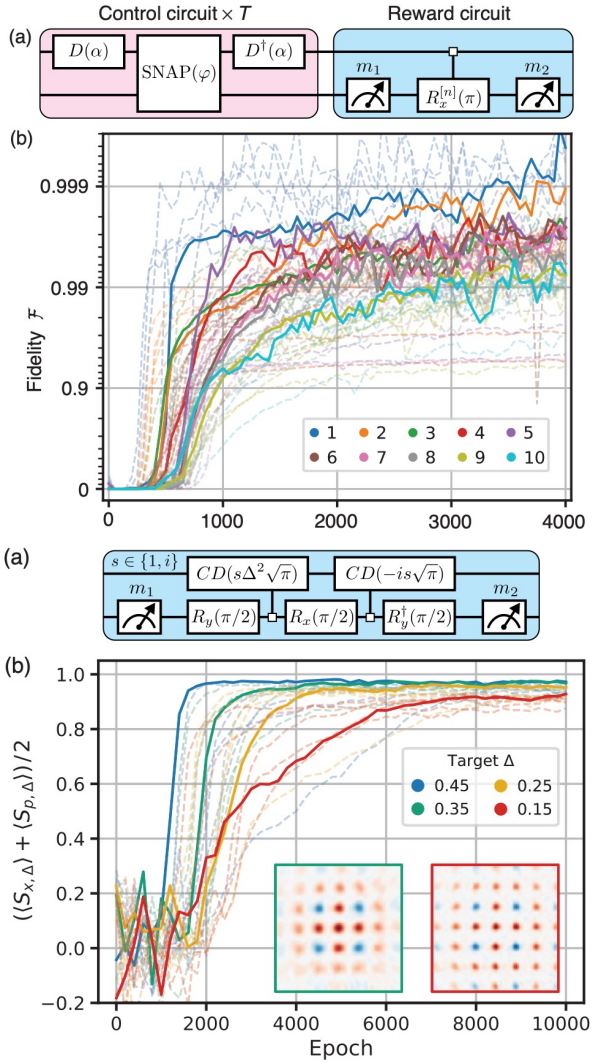
https://doi.org/10.1103/PhysRevX.12.011059



Quantum Control with RL

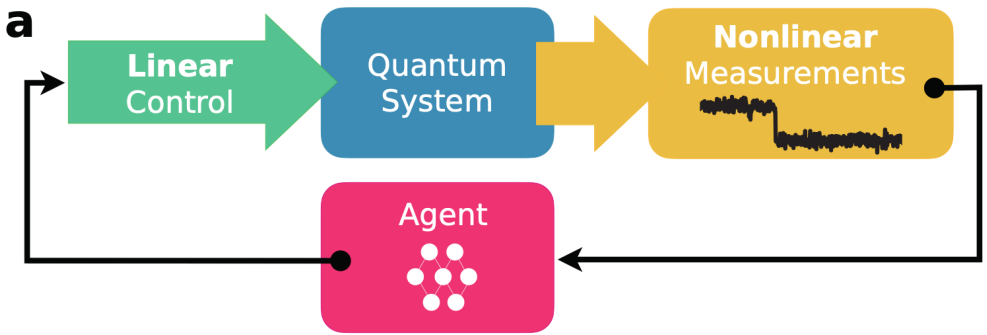
https://doi.org/10.1103/PhysRevX.12.011059

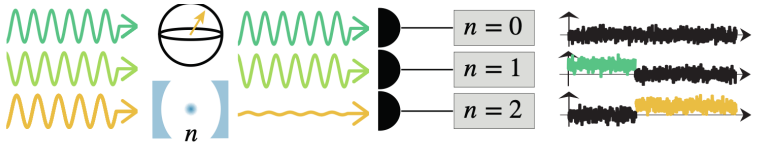


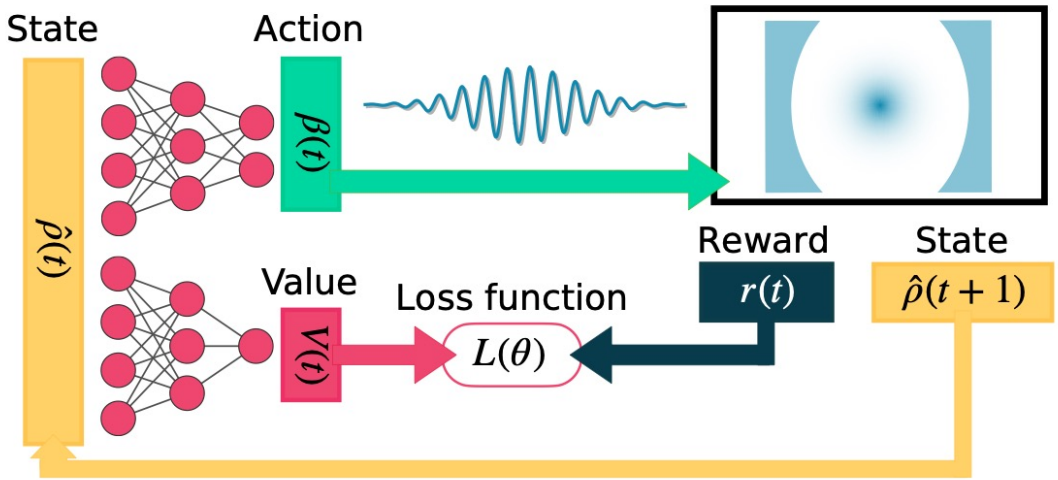


Quantum Control with RL

https://doi.org/10.22331/q-2022-06-28-747







Quantum Control with RL

https://doi.org/10.22331/q-2022-06-28-747

